

Core JAVA

UNIT 1

By Shivani Deopa

INTRODUCTION TO JAVA

- Java is a class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible.
- It is intended to let application developers write once, and run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation.
- Java was first released in 1995 and is widely used for developing applications for desktop, web, and mobile devices.
- Java is known for its simplicity, robustness, and security features, making it a popular choice for enterprise-level applications.

- JAVA was developed by James Gosling at Sun Microsystems Inc in the year 1995 and later acquired by Oracle Corporation. It is a simple programming language.
- Java makes writing, compiling, and debugging programming easy. It helps to create reusable code and modular programs. Java is a class- based, object-oriented programming language and is designed to have as few implementation dependencies as possible.
- A general-purpose programming language made for developers

to write once run anywhere that is compiled Java code can run on all platforms that support Java.

- Java applications are compiled to byte code that can run on any Java Virtual Machine. The syntax of Java is similar to c/c++.

Java Terminology

- Before learning Java, one must be familiar with these common terms of Java.
1. **Java Virtual Machine(JVM):** This is generally referred to as JVM. There are three execution phases of a program. They are written, compile and run the program.
 - Writing a program is done by a java programmer like you and me.
 - The compilation is done by the **JAVAC** compiler which is a primary Java compiler included in the Java development kit (JDK). It takes the Java program as input and generates bytecode as output.
 - In the Running phase of a program, **JVM** executes the bytecode generated by the compiler.
 - Every Operating System has a different JVM but the output they produce after the execution of bytecode is the same across all the operating systems. This is why Java is known as a **platform-independent language**.
 2. **Bytecode in the Development Process:** The Javac compiler of JDK compiles the java source code into bytecode so that it can be executed by JVM. It is saved as **.class** file by the compiler.

3. **Java Development Kit(JDK):** So, as the name suggests, it is a complete Java development kit that includes everything including compiler, Java Runtime Environment (JRE), java debuggers, java docs, etc. For the program to execute in java, we need to install JDK on our computer in order to create, compile and run the java program.
4. **Java Runtime Environment (JRE):** JDK includes JRE. JRE installation on our computers allows the java program to run, however, we cannot compile it. JRE includes a browser, JVM, applet support, and plugins. For running the java program, a computer needs JRE.
5. **Garbage Collector:** In Java, programmers can't delete the objects. To delete or recollect that memory JVM has a program called Garbage Collector. Garbage Collectors can recollect the objects that are not referenced. So Java makes the life of a programmer easy by handling memory management. However, programmers should be careful about their code whether they are using objects that have been used for a long time. Because Garbage cannot recover the memory of objects being referenced.
6. **ClassPath:** The classpath is the file path where the java runtime and Java compiler look for **.class** files to load. By default, JDK provides many libraries. If you want to include external libraries they should be added to the classpath.

Features of Java

- 1. Platform Independent:** Compiler converts source code to bytecode and then the JVM executes the bytecode generated by the compiler. This bytecode can run on any platform be it Windows, Linux, or macOS which means if we compile a program on Windows, then we can run it on Linux and vice versa. Each operating system has a different JVM, but the output produced by all the OS is the same after the execution of the bytecode. That is why we call java a platform-independent language.
- 2. Object-Oriented Programming Language:** Organizing the program in the terms of a collection of objects is a way of object-oriented programming, each of which represents an instance of the class.

The four main concepts of Object-Oriented programming are:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

3. Simple: Java is one of the simple languages as it does not have complex features like pointers, operator overloading, multiple inheritances, and Explicit memory allocation.

- 4. Robust:** Java language is robust which means reliable. It is developed in such a way that it puts a lot of effort into checking errors as early as possible, that is why the java compiler is able to detect even those errors that are not easy to detect by another programming language. The main features of java that make it robust are garbage collection, Exception Handling, and memory allocation.
- 5. Secure:** In java, we don't have pointers, so we cannot access out-of-bound arrays i.e it shows **ArrayIndexOutOfBoundsException** if we try to do so. That's why several security flaws like stack corruption or buffer overflow are impossible to exploit in Java. Also, java programs run in an environment that is independent of the os(operating system) environment which makes java programs more secure.
- 6. Distributed:** We can create distributed applications using the java programming language. Remote Method Invocation and Enterprise Java Beans are used for creating distributed applications in java. The java programs can be easily distributed on one or more systems that are connected to each other through an internet connection.
- 7. Multithreading:** Java supports multithreading. It is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of the CPU.
- 8. Portable:** As we know, java code written on one machine can be run on another machine. The platform-independent feature of java in which its platform-independent bytecode can be taken to any platform for execution makes java portable.

9. High Performance: Java architecture is defined in such a way that it reduces overhead during the runtime and at some times java uses Just In Time (JIT) compiler where the compiler compiles code on-demand basis where it only compiles those methods that are called making applications to execute faster.

10. Dynamic flexibility: Java being completely object-oriented gives us the flexibility to add classes, new methods to existing classes, and even create new classes through sub-classes. Java even supports functions written in other languages such as C, C++ which are referred to as native methods.

11. Sandbox Execution: Java programs run in a separate space that allows user to execute their applications without affecting the underlying system with help of a bytecode verifier. Bytecode verifier also provides additional security as its role is to check the code for any violation of access.

12. Write Once Run Anywhere: As discussed above java application generates a '.class' file that corresponds to our applications(program) but contains code in binary format. It provides ease of architecture-neutral ease as bytecode is not dependent on any machine architecture. It is the primary reason java is used in the enterprising IT industry globally worldwide.

13. Power of compilation and interpretation: Most languages are designed with the purpose of either they are compiled language or they are interpreted language. But java integrates arising enormous power as Java compiler compiles the source code to bytecode and JVM executes this bytecode to machine OS-dependent executable code.

Java programming format

```
// Importing classes from packages
import java.io.*;
// Main class
public class Demo
{
    // Main driver method
```

```
public static void main(String[] args)
{
    // Print statement
    System.out.println("My first Java
    Code");
}
}
```

Java Tokens

- A token is the smallest element of a program that is meaningful to the compiler. Tokens can be classified as follows:
 1. Keywords
 2. Identifiers
 3. Constants
 4. Special Symbols
 5. Operators
- Keyword:** Keywords are pre-defined or reserved words in a programming language. Each keyword is

meant to perform a specific function in a program. Since keywords are referred names for a compiler, they can't be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed.

- Java language supports following keywords:

abstract, boolean, case, catch, char, class, const, do, double, else, extends, float, for, goto, if, implements, import, int, interface, long, package, private, protected, public, return, static, switch, try, void, while etc

Identifiers: Identifiers are used as the general terminology for naming of variables, functions and arrays. These are user-defined names consisting of an arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character. Identifier names must differ in spelling and case from any keywords.

- You cannot use keywords as identifiers; they are reserved for special use. Once declared, you can use the identifier in later program statements to refer to the associated value. A special kind of identifier, called a statement label, can be used in goto statements.

- Examples of valid

identifiers : MyVariable

MYVARIABLE

myvariable

x

x1

i1

_myvariable

\$myvariable

sum_of_array

Special Symbols: The following special symbols are used in Java having some special meaning and thus, cannot be used for some other purpose. Eg- [] () {}, ; * =

1. Brackets[]: Opening and closing brackets are used as array element reference. These indicate single and multidimensional subscripts.
2. Parentheses(): These special symbols are used to indicate function calls and function parameters.
3. Braces{}: These opening and ending curly braces marks the start and end of a block of code containing more than one executable statement.
4. comma (,): It is used to separate more than one statements like for separating parameters in function calls.
5. semi colon : It is an operator that essentially invokes something called an initialization list.

6. asterick (*): It is used to create pointer variable.

Constants/Literals: Constants are also like normal variables. But, the only difference is, their values can not be modified by the program once they are defined. Constants refer to fixed values. They are also called as literals. Constants may belong to any of the data type.

- Syntax: final data_type variable_name;

Operators: Java provides many types of operators which can be used according to the need. They are classified based on the functionality they provide.

Some of the types are-

- Arithmetic Operators
- Unary Operators

- Assignment Operator
- Relational Operators
- Logical Operators
- Bitwise Operators etc

Java Statements

- In Java, a **statement** is an executable instruction that tells the compiler what to perform.
- It forms a complete command to be executed and can include one or more expressions. A sentence forms a complete idea that can include one or more clauses.

Types of Statements

1. Expression Statements
2. Declaration Statements
3. Control Statements

Expression Statements

- Expression is an essential building block of any Java program. Generally, it is used to generate a new value. Sometimes, we can also assign a value to a variable. In Java, expression is the combination of values, variables, operators, and method calls.

There are three types of expressions in Java:

1. Expressions that **produce** a value. For example, **(6+9), (9%2),**

(pi*radius) + 2. Note that the expression enclosed in the parentheses will be evaluate first, after that rest of the expression.

2. Expressions that **assign** a value. For example, **number = 90, pi = 3.14**.

- Expression that **neither produces any result nor assigns a value**. For example, **increment** or **decrement** a value by using increment or decrement operator respectively, **method invocation**, etc.
- These expressions modify the value of a variable or state (memory) of a program. For example, **count++**, **int sum = a + b**; The expression changes

only the value of the variable **sum**. The value of variables **a** and **b** do not change, so it is also a side effect.

Declaration Statements

- In declaration statements, we declare variables and constants by specifying their data type and name. A variable holds a value that is going to use in the Java program. For example:

```
int quantity;
```

```
boolean flag;
```

```
String message;
```

- Java also allows us to declare multiple variables in a single declaration statement. Note that all the variables must be of the same data type.

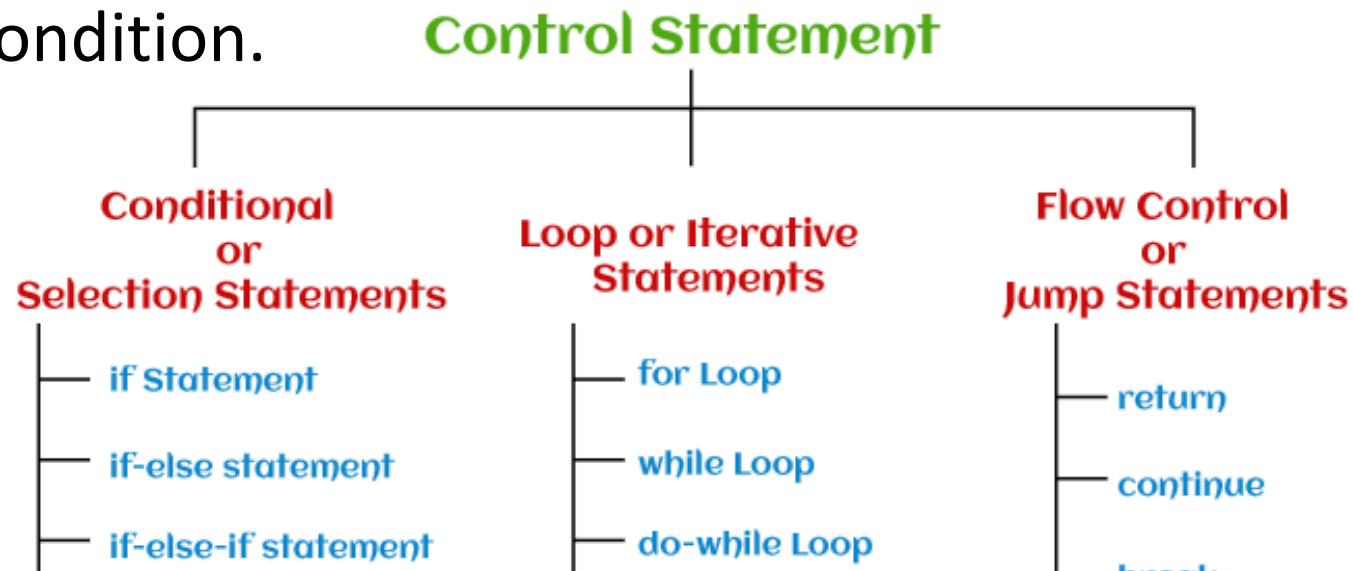
```
int quantity, batch_number, lot_number;
```

```
boolean flag = false, isContains = true;
```

```
String message = "Hello", how are you;
```


Control Statement

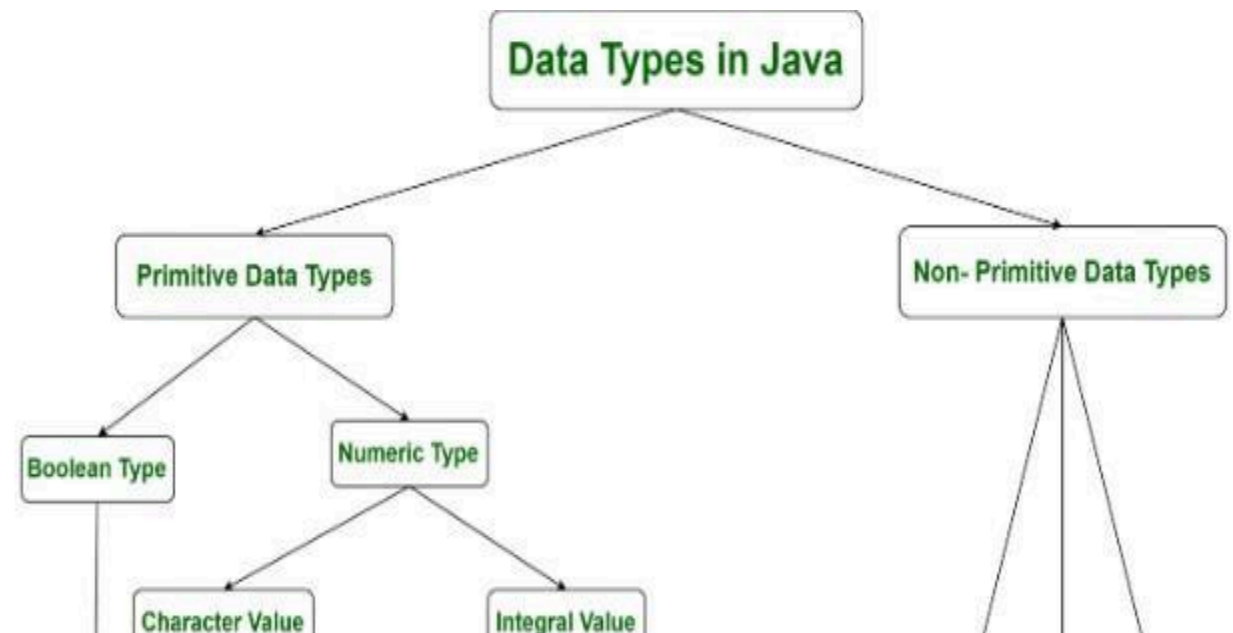
- Control statements decide the flow (order or sequence of execution of statements) of a Java program. In Java, statements are parsed from top to bottom.
- Therefore, using the control flow statements can interrupt a particular section of a program based on a certain condition.



Java Data Types

- Data types in Java are of different sizes and values that can be stored in the variable that is made as per convenience and circumstances to cover up all test cases.
- Java has two categories in which data types are segregated

1. Primitive Data Type: such as

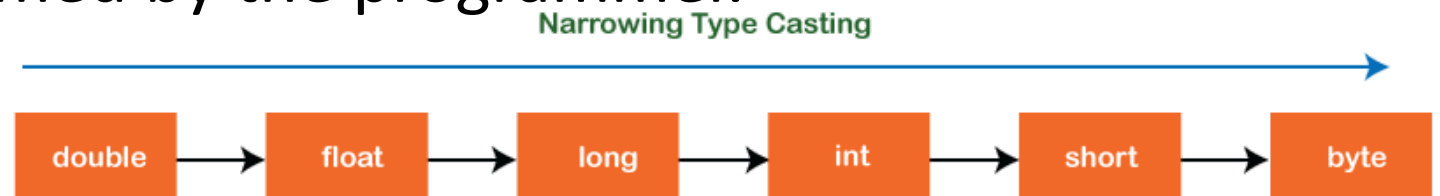


boolean, char, int, short, byte,
long, float, and double

- 2. Non-Primitive Data Type or Object Data type:** such as String, Array, etc.

Typecasting

- Type casting is a method or process that converts a data type into another data type in both ways manually and automatically.
- The automatic conversion is done by the compiler and manual conversion performed by the programmer.
- Types of Type Casting:



1. Widening Type Casting
2. Narrowing Type Casting

Widening Type Casting

- Converting a lower data type into a higher one is called **widening** type casting. It is also known as **implicit conversion** or **casting down**. It is done automatically. It is safe because there is no chance to lose data. It takes place when:
- Both data types must be compatible with each other.
- The target type must be larger than the source type.

byte -> short -> char -> int -> long -> float -> double

Narrowing Type Casting

- Converting a higher data type into a lower one is called narrowing type casting. It is also known as explicit conversion or casting up. It is done manually by the programmer. If we do not perform casting then the compiler reports a compile-time error.

double -> float -> long -> int -> char -> short -> byte

Arrays

- **Array in java** is a group of like-typed variables referred to by a common name. Arrays in Java work differently than they do in C/C++. Following are some important points about Java arrays.
- In Java, all arrays are dynamically allocated. Arrays are stored in contiguous memory [consecutive memory locations].
- Since arrays are objects in Java, we can find their length using the object property *length*. This is different from C/C++, where we find length using `sizeof`.
- A Java array variable can also be declared like other variables with `[]` after the data type.
- The variables in the array are ordered, and each has an index beginning with 0.

- Java array can also be used as a static field, a local variable, or a method parameter.
- The **size** of an array must be specified by int or short value and not long.
- Every array type implements the interfaces Cloneable and java.io.Serializable.
- This storage of arrays helps us randomly access the elements of an array [Support Random Access].
- The size of the array cannot be altered(once initialized). However, an array reference can be made to point to another array.

- An array can contain primitives (int, char, etc.) and object (or non-primitive) references of a class depending on the definition of the array. In the case of primitive data types, the actual values are stored in contiguous memory locations. In the case of class objects, the actual objects are stored in a heap segment.

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

One-Dimensional Arrays:

- The general form of a one-dimensional array declaration is
- `type var-name[];` OR `type[] var-name;`
- An array declaration has two components: the type and the name. type declares the element type of the array. The element type determines the data type of each element that comprises the array. Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc., or user-defined data types (objects of a class). Thus, the element type for the array determines what type of data the array will hold.
- Example:

// both are valid declarations

`int intArray[];` or `int[] intArray;`

`byte byteArray[];`

`short shortsArray[];`

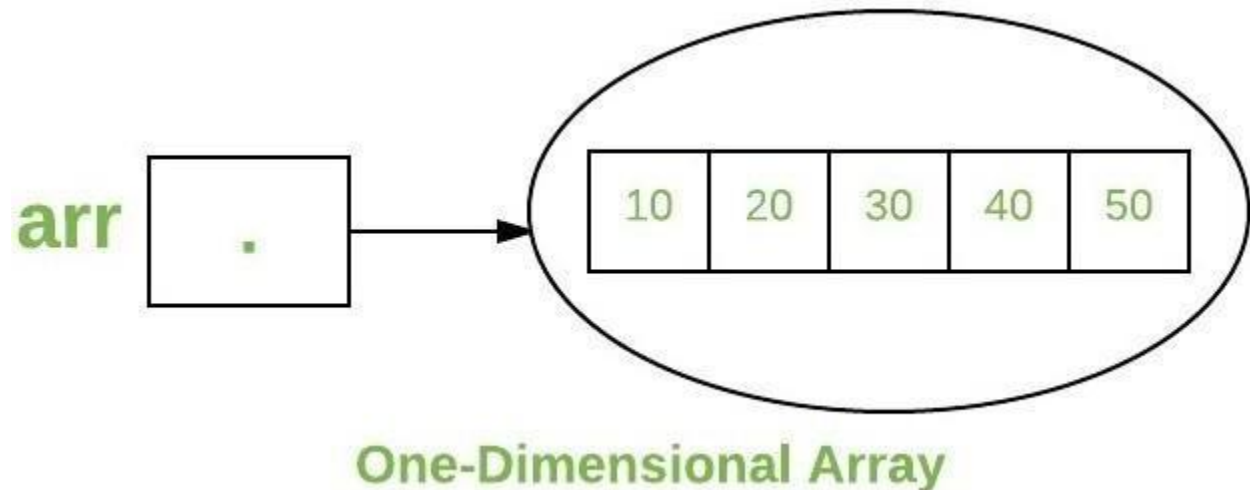
`boolean booleanArray[];`

`long longArray[];`

`float floatArray[];`

`double doubleArray[];`

`char charArray[];`



- **Multidimensional Arrays:**

- Multidimensional arrays are **arrays of arrays** with each element of the array holding the reference of other arrays. These are also known as Jagged Arrays. A multidimensional array is created by appending one set of square brackets ([]) per dimension.

- **Syntax :**

datatype [][] arrayrefvariable;

or

datatype

arrayrefvariable[][]; import java.io.*;

class multiarray {

public static void main (String[] args) {

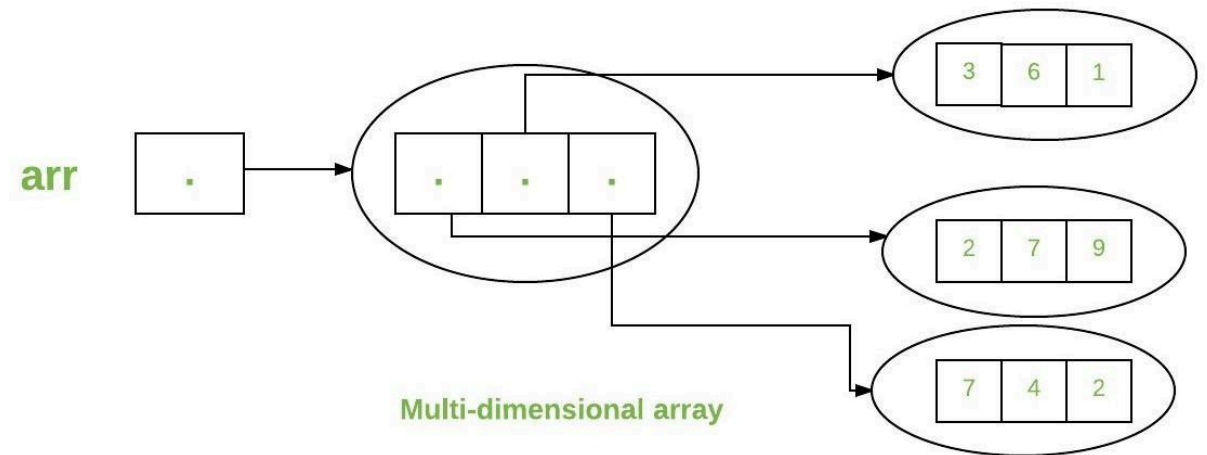
// Syntax

int [][] arr= new int[3][3];

// 3 row and 3 column

}

}



OOPS

- **Object-Oriented Programming** or OOPs refers to languages that use objects in programming, they use objects as a primary source to implement what is to happen in the code. Objects are seen by the viewer or user, performing tasks assigned by you. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism etc. in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.
- Let us discuss prerequisites by polishing concepts of method declaration and message

passing. Starting off with the method declaration, it consists of six components:

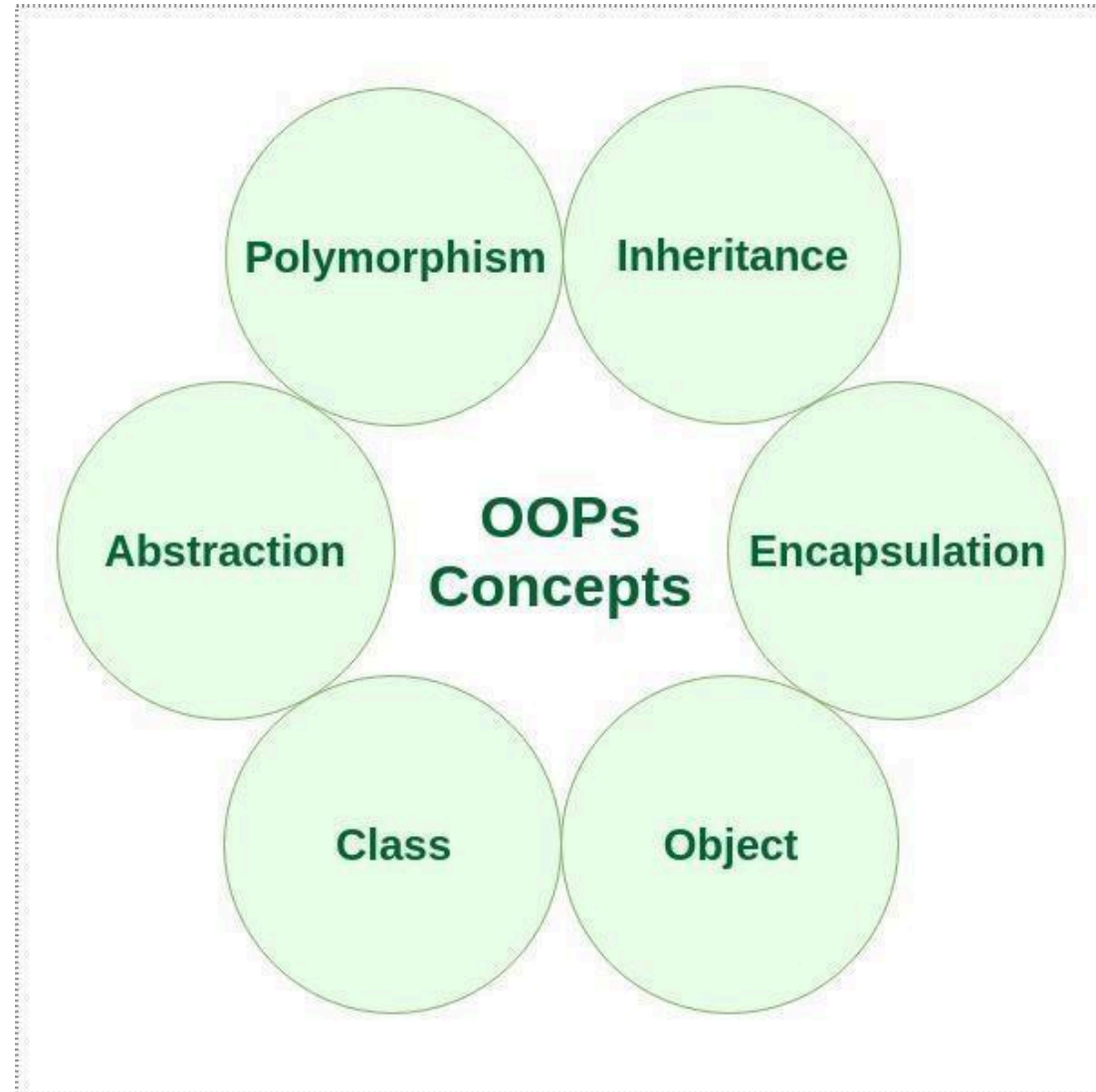
- **Access Modifier:** Defines the **access type** of the method i.e. from where it can be accessed in your application. In Java, there are 4 types of access specifiers:
 1. **public:** Accessible in all classes in your application.
 2. **protected:** Accessible within the package in which it is defined and in its subclass(es) (including subclasses declared outside the package).

3. **private:** Accessible only within the class in which it is defined.
4. **default (declared/defined without using any modifier):** Accessible within the same class and package within which its class is defined.

- **The return type:** The data type of the value returned by the method or void if it does not return a value.
- **Method Name:** The rules for field names apply to method names as well, but the convention is a little different.
- **Parameter list:** Comma-separated list of the input parameters that are defined, preceded by their data type, within the enclosed parentheses. If there are no parameters, you must use empty parentheses ().
- **Exception list:** The exceptions you expect the method to throw. You can specify these exception(s).
- **Method body:** It is the block of code, enclosed between braces, that you need to execute to perform your intended operations.

OOPS concepts are as follows:

- Class
- Object
- Method and method passing
- Pillars of OOPs
 - Abstraction
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Compile-time polymorphism
 - Runtime polymorphism



Classes and Object

A class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times. This includes classes for objects occurring more than once in your code. In general, class declarations can include these components in order:

- **Modifiers:** A class can be public or have default access
- **Class name:** The class name should begin with the initial letter capitalized by convention.

- **Superclass (if any):** The name of the class's parent (superclass), if any, preceded by the keyword `extends`. A class can only extend (subclass) one parent.
- **Interfaces (if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword `implements`. A class can implement more than one interface.
- **Body:** The class body is surrounded by braces, `{ }`.

An object is a basic unit of Object-Oriented Programming that represents real-life entities. A typical Java program creates many objects, which as you know, interact by invoking methods. The objects are what perform your code, they are the part of your code visible to the viewer/user. An object mainly consists of:

- **State:** It is represented by the attributes of an object. It also reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object. It also reflects the response of an object to other objects.
- **Identity:** It is a unique name given to an object that enables it to interact with other objects.
- **Method:** A method is a collection of statements that perform some specific task and return the result to the caller. A method can perform some specific task without returning anything. Methods allow us to **reuse** the code without retyping it, which is why they are considered **time savers**. In Java, every method must be part of some class, which is different from languages like C, C++, and Python.

Static Keywords

- The **static keyword** in Java is mainly used for memory management. The static keyword in Java is used to share the same variable or method of a given class.
- The users can apply static keywords with variables, methods, blocks, and nested classes. The static keyword belongs to the class than an instance of the class.

- The static keyword is used for a constant variable or a method that is the same for every instance of a class.
- **The *static* keyword is a non-access modifier in Java that is applicable for the following:**
 1. Blocks
 2. Variables
 3. Methods
 4. Classes

Characteristics of static keyword:

- Here are some characteristics of the static keyword in Java:
 1. **Shared memory allocation:** Static variables and methods are allocated memory space only once during the execution of the program. This memory space is shared among all instances of the class, which makes static members useful for maintaining global state or shared functionality.
 2. **Accessible without object instantiation:** Static members can be accessed without the need to create an instance of the class. This makes them useful for providing utility functions and constants that can be used across the entire program.
 3. **Associated with class, not objects:** Static members are associated with the class, not with individual objects. This means that changes to a static member are reflected in all instances of the class, and that you can access static members using the class name rather than an object reference.
 4. **Cannot access non-static members:** Static methods and variables cannot access non-static members of a class, as they are not associated with any particular instance of the class.
 5. **Can be overloaded, but not overridden:** Static methods can be overloaded, which means that you can define multiple methods with the same name but different parameters. However, they cannot be overridden, as they are associated with the class rather than with a particular instance of the class.

Static blocks

- If you need to do the computation in order to initialize your **static variables**, you can declare a static block that gets executed exactly once, when the class is first loaded.

```
class Test
{
    //      static
    variable
    static int a =
    10; static int
    b;
    //  static
    block
    static {
        System.out.println("Static      block
        initialized."); b = a * 4;
    }

    public static void main(String[] args)
    {
        System.out.println("from main");
        System.out.println("Value of a : "+a);
        System.out.println("Value of b : "+b);
    }
}
```

}

}

Static variables

- When a variable is declared as static, then a single copy of the variable is created and shared among all objects at the class level. Static variables are, essentially, global variables. All instances of the class share the same static variable.

Important points for static variables:

- We can create static variables at the class level only.
- static block and static variables are executed in the order they are present in a program.

// Java program to demonstrate execution of static blocks and variables class Test

```
{
    // static
    variable
    static int
    a = m1();
    //
    static
    block
    static
    {
        System.out.println("Inside static block");
    }
    // static
    method
    static
    int m1()
    {
        System.out.println("from
        m1"); return 20;
    }
    // static method(main !!)
    public static void main(String[] args)
    {
```

```
System.out.println("Value of a : "+a);
```

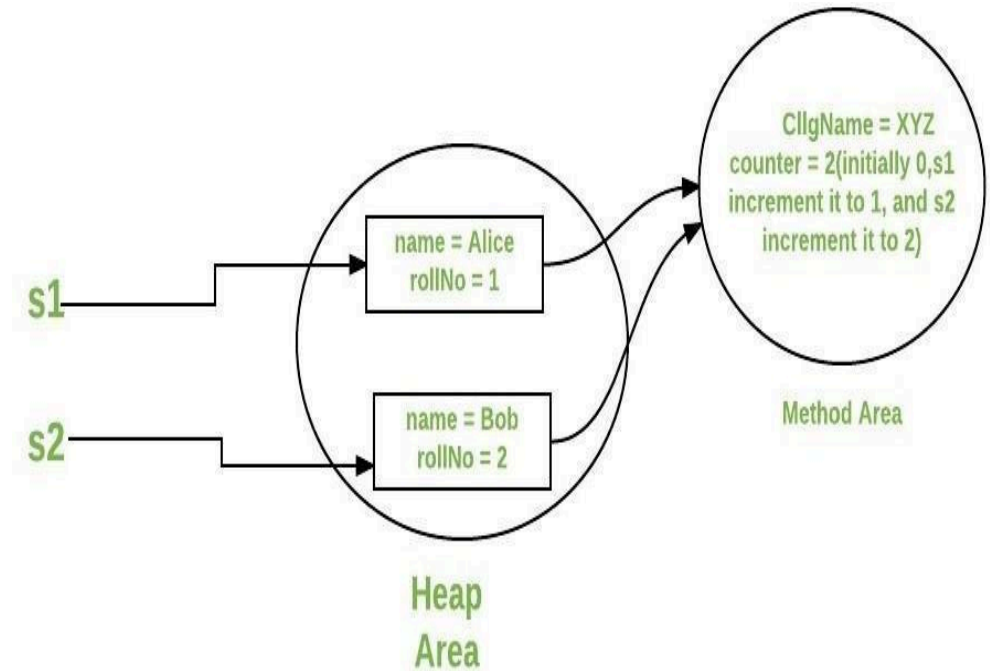
```
System.out.println("from main");
```

```
}
```

```
}
```

When to use static variables and methods?

- Use the static variable for the property that is common to all objects.
- For example, in class Student, all students share the same college name. Use static methods for changing static variables.
- Consider the following java program, that illustrates the use of static keywords with variables and methods.



```

class Student { String
    name; int rollNo;
    // static variable
    static String cllgName;
    // static counter to set unique roll no
    static int counter = 0;
    public Student(String name)
    {
        this.name = name;
        this.rollNo = setRollNo();
    }
    // getting unique rollNo through static variable(counter) static int
    setRollNo()
    {
        counter++; return counter;
    }
    // static method

```

```

static void setCllg(String name) { cllgName = name; }
    // instance method
    void getStudentInfo()
    {
        System.out.println("name : " + this.name);
        System.out.println("rollNo : " + this.rollNo);
        // accessing static variable
        System.out.println("cllgName : " + cllgName);
    }
// Driver class
public class StaticDemo {
    public static void main(String[] args)
    {
        // calling static method
        // without instantiating
        Student class
        Student.setCllg("XYZ");
        Student s1 = new
        Student("Alice");
        Student s2 = new
        Student("Bob");
        s1.getStudentInfo();
    }
}

```

```
s2.getStudentInfo();  
}  
}
```

Constructors

- In Java, Constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling the constructor, memory for the object is allocated in the memory. It is a special type of method that is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called.

How Java Constructors are Different From Java Methods?

- Constructors must have the same name as the class within which it

is defined it is not necessary for the method in Java.

- Constructors do not return any type while method(s) have the return type or **void** if does not return any value.
- Constructors are called only once at the time of Object creation while method(s) can be called any number of times.

When Constructor is called?

- Each time an object is created using a **new()** keyword, at least one constructor (it could be the default constructor) is invoked to assign initial values to the **data members** of the same class. Rules for writing constructors are as follows:
- The constructor(s) of a class must have the same name as the class name in which it resides.
- A constructor in Java can not be abstract, final, static, or Synchronized.
- Access modifiers can be used in constructor declaration to control its access i.e which other class can call the constructor.

Types of Constructors in Java

1. Default Constructor
2. Parameterized Constructor

1. Default Constructor in Java

constructor.

- A constructor that has no parameters is known as default the constructor.
- A default constructor is invisible. And if we write a constructor with no arguments, the compiler does not create a default constructor. It is taken out.
- It is being overloaded and called a parameterized constructor. The default constructor changed into the parameterized constructor.
- But Parameterized constructor can't change the default

```
// Java Program to demonstrate
// Default
Constructor import
java.io.*;
// Driver class
class ABC {

    // Default
    Constructor ABC()
    {
        System.out.println("Default constructor");
    }
    // Driver function
    public static void main(String[] args)
    {
        ABC obj = new ABC();
    }
}
```

2. Parameterized Constructor in Java

- A constructor that has parameters is known as parameterized constructor.
- If we want to initialize fields of the class with our own values, then use a parameterized constructor.

```
import
java.io.*;
class
ABC {
    Strin
    g
    name
    ; int
    id;
    ABC(String name, int id)
    {
        this.name
        = name;
        this.id = id;
    }
}
class GFG {
    public static void main(String[] args)
    {
```

```
// This would invoke the parameterized constructor.
```

```
ABC obj1 = new ABC("avinash", 68);
```

```
System.out.println("Name :" + obj1.name + " and ID :" + obj1.id);
```

```
}
```

```
}
```

this Key Word

- In Java, 'this' is a reference variable that refers to the current object, or can be said "this" in Java is a keyword that refers to the current object instance. It can be used to call current class methods and fields, to pass an instance of the current class as a parameter, and to differentiate between the local and instance variables. Using "this" reference can improve code readability and reduce naming conflicts.
- **Methods to use 'this' in Java**
 1. Using the 'this' keyword to refer to current class instance variables.
 2. Using this() to invoke the current class constructor

3. Using 'this' keyword to return the current class instance
 4. Using 'this' keyword as the method parameter
 5. Using 'this' keyword to invoke the current class method
 6. Using 'this' keyword as an argument in the constructor call
- Below is the implementation of this reference:

```
public class ThisExample {  
  
    int num = 10;  
    public ThisExample() {  
  
        System.out.println("Inside constructor");  
    }  
  
    public ThisExample(int num)  
    {  
        // Invoking default constructor  
        this();  
        // Assigning the local variable num to  
        the instance  
        this.num = num;  
    }  
}
```

```
void show() {  
  
    System.out.println("Inside  
show method");  
}  
  
    public static void main(String[] args)  
    {  
        ThisExample obj =  
            new ThisExample(100);  
        obj.display();  
    }  
}
```

}

Advantages of using 'this' reference

- There are certain advantages of using 'this' reference in Java as mentioned below:
- It helps to distinguish between instance variables and local variables with the same name.
- It can be used to pass the current object as an argument to another method.
- It can be used to return the current object from a method.
- It can be used to invoke a constructor from another overloaded constructor in the same class.

Disadvantages of using 'this' reference

- Although 'this' reference comes with many advantages there are certain disadvantages of also:
- Overuse of this can make the code harder to read and understand.
- Using this unnecessarily can add unnecessary overhead to the program.
- Using this in a static context results in a compile-time error.

- Overall, this keyword is a useful tool for working with objects in Java, but it should be used judiciously and only when necessary.

Inheritance

- Inheritance is an important pillar of OOP(Object-Oriented Programming).
- It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class.
- In Java, Inheritance means creating new classes based on existing ones.

- A class that inherits from another class can reuse the methods and fields of that class.
- In addition, you can add new fields and methods to your current class as well.

Why Do We Need Java Inheritance?

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.
- **Method Overriding:** Method Overriding is achievable only through Inheritance. It is one of the ways by which Java achieves Run Time Polymorphism.
- **Abstraction:** The concept of abstract where we do not have to provide all details is achieved through inheritance. Abstraction only shows the functionality to the user.

Important Terminologies Used in Java Inheritance

- **Class:** Class is a set of objects which shares common characteristics/ behavior and common properties/ attributes. Class is not a real-world entity. It is just a template or blueprint or prototype from which objects are created.
- **Super Class/Parent Class:** The class whose features are inherited is known as a superclass(or a base class or a parent class).
- **Sub Class/Child Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

How to Use Inheritance in Java?

- The extends keyword is used for inheritance in Java. Using the extends keyword indicates you are derived from an existing class.
- In other words, “extends” refers to increased functionality.
- Syntax :

```
class derived-class extends base-class
{
    //methods and fields
```

}

Java Inheritance Types

- Below are the different types of inheritance which are supported by Java.

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

Single Inheritance

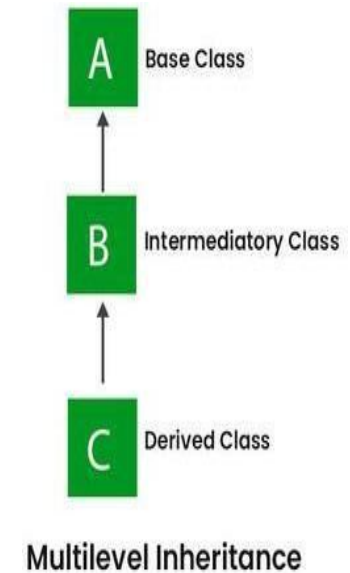
- In single inheritance, subclasses inherit the features of one superclass. In the image below, class A serves as a base class for the



derived class B.

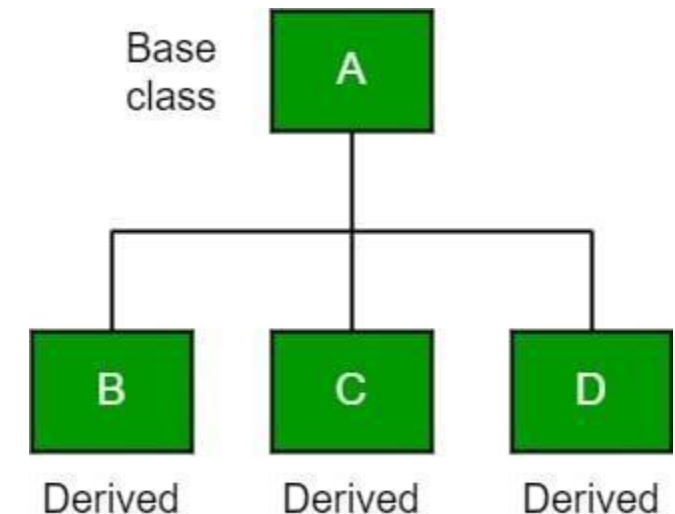
Multilevel Inheritance

- In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the grandparent's members.



Hierarchical Inheritance

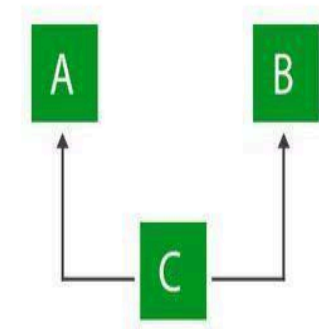
- In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as



a base class for the derived classes B, C, and D.

Multiple Inheritance (Through Interfaces)

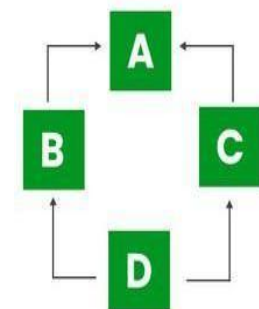
- In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. Please note that Java does **not** support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces. In the image below, Class C is derived from interfaces A and B.



Multiple Inheritance

Hybrid Inheritance(Through Interfaces)

- It is a mix of two or more of the above types of inheritance. Since Java doesn't support multiple inheritances with classes, hybrid inheritance is also not possible with classes. In Java, we can achieve



Hybrid Inheritance

hybrid inheritance only through Interfaces.

super Key Word

- The **super** keyword in Java is a reference variable that is used to refer to parent class objects
- The keyword “super” came into the picture with the concept of

Inheritance.

- It is majorly used in the following contexts as mentioned below:
 1. Use of super with variables
 2. Use of super with methods
 3. Use of super with constructors

Characteristics of super Keyword in Java

- In Java, the super keyword is used to refer to the parent class of a subclass. Here are some of its characteristics:
- super is used to call a superclass constructor: When a subclass is created, its constructor must call the constructor of its parent class. This is done using the super() keyword, which calls the constructor of the parent class.
- super is used to call a superclass method: A subclass can call a method defined in its parent class using the super keyword. This is useful when the subclass wants to invoke the parent class's implementation of the method in addition to its own.
- super is used to access a superclass field: A subclass can access a field defined in its parent class using the super keyword. This is useful when the subclass wants to reference the parent class's version of a field.
- super must be the first statement in a constructor: When calling a superclass constructor, the super() statement must be the first statement in the constructor of the subclass.
- super cannot be used in a static context: The super keyword cannot be used in a static context, such as in a static method or a static variable initializer.

- super is not required to call a superclass method: While it is possible to use the super keyword to call a method in the parent class, it is not required. If a method is not overridden in the subclass, then calling it without the super keyword will invoke the parent class's implementation.

Use of super with Variables

- This scenario occurs when a derived class and base class has the same data members. In that case, there is a possibility of ambiguity for the JVM.
- In the example, both the base class and subclass have a member maxSpeed. We could access maxSpeed of base class in subclass using super keyword.

```
class Vehicle {  
    int maxSpeed = 120;  
}  
  
class Car extends  
    Vehicle { int  
    maxSpeed = 180;  
    void display()  
    {  
  
        System.out.println("Maximum Speed: " + super.maxSpeed);  
    }  
}  
  
// Driver  
Program class  
Test {  
    public static void main(String[] args)  
  
    {  
  
        Car small = new
```

Car();

small.display();

}

}

Use of super with Methods

- This is used when we want to call the parent class method. So whenever a parent and child class have the same-named methods then to resolve ambiguity we use the super keyword.

- In the example, we have seen that

if we only call method message()
then, the current class message() is

```
class
    Person {
        void
        message()
        { System.out.println("This is person class\n"); }
    }
class Student extends Person {
    void message()
    { System.out.println("This is student
    class");
        } void display()
    {
        message();
        super.message();
    }
}
```

invoked but with the use of the super
keyword, message() of the superclass

could also be invoked.

```
class Test {  
    public static void main(String args[])  
    {  
        Student s = new  
        Student(); s.display();  
    }  
}
```

Use of super with constructors

- The super keyword can also be used to access the parent class constructor. One more important thing is that 'super' can call both parametric as well as non-parametric constructors depending on the situation.

```
class
    Pers
    on {
        Pers
        on()
        {
            System.out.println("Person class Constructor");
        }
    }
class Student extends Person {
    Student()
    {
        super();
        System.out.println("Student class Constructor");
    }
}
class Test {
    public static void main(String[] args)
    {
        Student s = new Student();
    }
}
```

}

Advantages of Using Java super keyword

- The super keyword in Java provides several advantages in object-oriented programming:
 1. **Enables reuse of code:** Using the super keyword allows subclasses to inherit functionality from their parent classes, which promotes the reuse of code and reduces duplication.
 2. **Supports polymorphism:** Because subclasses can override methods and access fields from their parent classes using super, polymorphism is possible. This allows for more flexible and extensible code.
 3. **Provides access to parent class behavior:** Subclasses can access and use methods and fields defined in their parent classes through the super keyword, which allows them to take advantage of existing behavior without having to re-implement it.
 4. **Allows for customization of behavior:** By overriding methods and using super to call the parent implementation, subclasses can customize and extend the behavior of their parent classes.
 5. **Facilitates abstraction and encapsulation:** The use of super promotes encapsulation and abstraction by allowing subclasses to focus on their own behavior while relying on the parent class to handle lower-level

details.

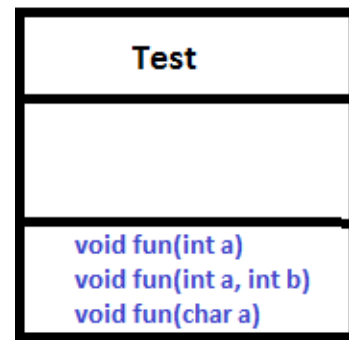
Polymorphism (overloading and overriding)

- Polymorphism is considered one of the important features of Object-Oriented Programming. Polymorphism allows us to perform a single action in different ways.
- In other words, polymorphism allows you to define one interface and have multiple implementations. The word “poly” means many and “morphs” means forms, So it means many forms.

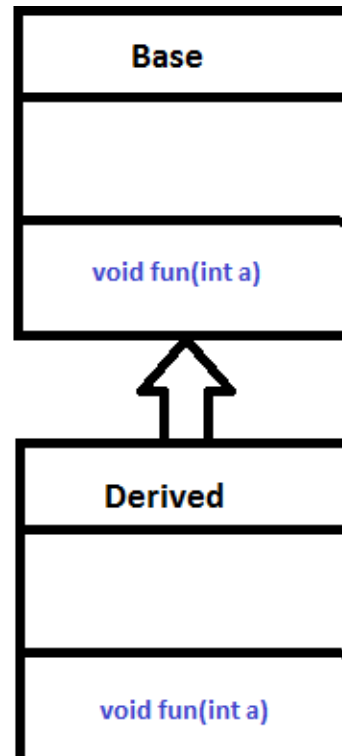
- In Java polymorphism is mainly divided into two types:
 1. Compile-time Polymorphism
 2. Runtime Polymorphism

Compile-Time Polymorphism

- It is also known as static polymorphism. This type of polymorphism is achieved by function.



Overloading



Method Overloading

- When there are multiple functions with the same name but different parameters then these functions are said to be overloaded.
- Functions can be overloaded by changes in the number of arguments or/and a change in the type of arguments.

```
class Helper {  
    static int Multiply(int a, int b)  
    {  
        return a * b;  
    }  
    static double Multiply(double a, double b)  
    {  
        return a * b;  
    }  
}  
class GFG {  
    public static void main(String[] args)  
    {  
  
        System.out.println(Helper.Multiply(2, 4));  
        System.out.println(Helper.Multiply(5.5, 6.3));  
    }  
}
```

Runtime Polymorphism

- It is also known as Dynamic Method Dispatch. It is a process in which a function call to the overridden method is resolved at Runtime. This type of polymorphism is achieved by Method Overriding.
- Method overriding, on the other hand, occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be overridden.

```
class
    Parent {
        void
        Print
        ()
        {
            System.out.println("parent class");
        }
    }
class subclass1 extends Parent {
    void Print() { System.out.println("subclass1"); }
}
class subclass2 extends
    Parent { void Print()
    {
        System.out.println("subclass2");
    }
}
class GFG {
    public static void main(String[] args)
    {
        Parent a;
        a = new subclass1();
        a.Print();
        a = new
```

```
subclass2();
```

```
a.Print();
```

```
}
```

```
}
```


Advantages of Polymorphism in Java

- Increases code reusability by allowing objects of different classes to be treated as objects of a common class.
- Improves readability and maintainability of code by reducing the amount of code that needs to be written and maintained.
- Supports dynamic binding, enabling the correct method to be called at runtime, based on the actual class of the object.
- Enables objects to be treated as a single type, making it easier to write generic code that can handle objects of different types.

Disadvantages of Polymorphism in Java

- Can make it more difficult to understand the behavior of an object, especially if the code is complex.
- This may lead to performance issues, as polymorphic behavior may require additional computations at runtime.

Interfaces

- An Interface in Java programming language is defined as an abstract type used to specify the behavior of a class. An interface in Java is a blueprint of a behaviour. A Java interface contains static constants and abstract methods.
- The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not the method body. It is used to achieve abstraction and multiple inheritance in Java. In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

- When we decide a type of entity by its behaviour and not via attribute we should define it as an interface.
- Like a class, an interface can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, no body).

- Interfaces specify what a class must do and not how. It is the blueprint of the behaviour.
- Interface do not have constructor.
- Represent behaviour as interface unless every sub-type of the class is guarantee to have that behaviour.
- An Interface is about capabilities like a Player may be an interface and any class implementing Player must be able to (or must implement) move(). So it specifies a set of methods that the class has to implement.
- If a class implements an interface and does not provide method bodies for all functions specified in the interface, then the class must be declared abstract.
- To declare an interface, use the interface keyword. It is used to provide total abstraction. That means all the methods in an interface are declared with an empty body and are public and all fields are public, static, and final by

default.

- A class that implements an interface must implement all the methods declared in the interface. To implement interface use implements keyword.

Why do we use an Interface?

- It is used to achieve total abstraction.
- Since java does not support multiple inheritances in the case of class, by using an interface it can achieve multiple inheritances.
- Any class can extend only 1 class but can any class implement infinite number of interface.
- It is also used to achieve loose coupling.
- Interfaces are used to implement abstraction. So the question arises why use interfaces when we have abstract classes?
- The reason is, abstract classes may contain non-final variables, whereas variables in the interface are final, public and static.

- The major differences between a class and an interface are:

```
import
java.io.*;
interface In1
{
    // public, static and final
    final int a = 10;
    // public and
    abstract void
    display();
}
class TestClass implements In1 {
    public void display(){
        System.out.println("HELLO");
    }
    public static void main(String[] args)
    {
        TestClass t = new TestClass();
        t.display();
        System.out.println(a);
    }
}
```

}

}

Abstraction

- In Java, abstraction is achieved by interfaces and abstract classes. We can achieve 100% abstraction using interfaces.
- Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details.
- The properties and behaviors of an object differentiate it from

other objects of similar type and also help in classifying/grouping the objects.

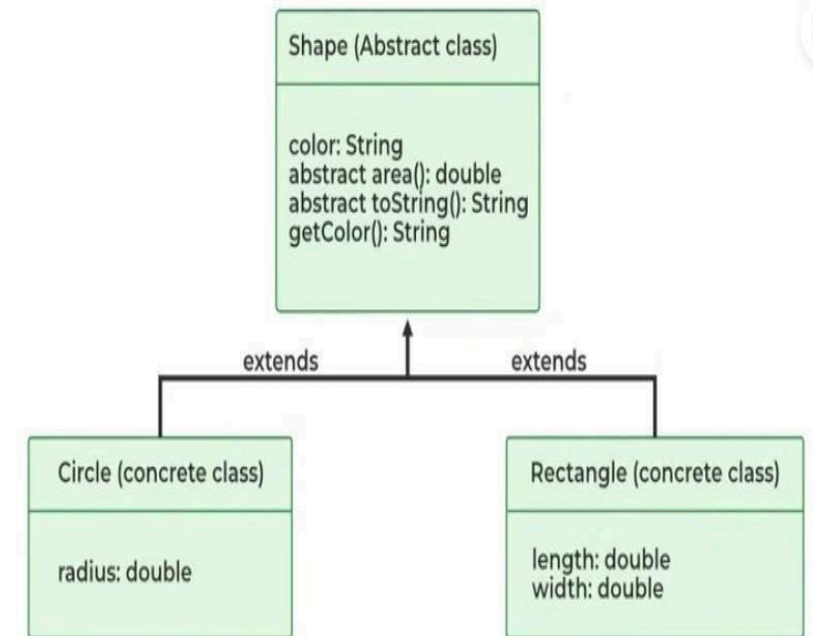
Java Abstract classes and Java Abstract methods

- An abstract class is a class that is declared with an abstract keyword.
- An abstract method is a method that is declared without implementation.
- An abstract class may or may not have all abstract methods. Some of them can be concrete methods
- A method-defined abstract must always be redefined in the subclass, thus making overriding compulsory or making the subclass itself abstract.
- Any class that contains one or more abstract methods must also be declared with an abstract keyword.
- There can be no object of an abstract class. That is, an abstract class can not be directly instantiated with the new operator.
- An abstract class can have parameterized constructors and the default constructor is always present in an abstract class.

When to use abstract classes and abstract methods

S. No.	Class	Interface
1.	In class, you can instantiate variables and create an object.	In an interface, you can't instantiate variables and create an object.
2.	Class can contain concrete(with implementation) methods	The interface cannot contain concrete(with implementation) methods
3.	The access specifiers used with classes are private, protected, and public.	In Interface only one specifier is used- Public.

- There are situations in which we will want to define a superclass that declares the structure of a given abstraction



Abstract Class in Java

without providing a complete implementation of every method. Sometimes we will want to create a superclass that only defines a generalization form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details.

- Consider a classic “shape” example, perhaps used in a computer-aided design system or game simulation. The base type is “shape” and each shape has a color, size, and so on. From this, specific types of shapes are derived (inherited) — circle, square, triangle, and so on — each of which may have additional characteristics and behaviors. For example, certain shapes can be flipped. Some behaviors may be different, such as when you want to calculate the area of a shape. The type hierarchy embodies both the similarities and differences between the shapes.

```
    abstract class Animal {
        private String name;
    public Animal(String name)
        { this.name = name; }
    public abstract void makeSound();
    public String getName()
        { return name; }
}
class Dog extends Animal {
    public Dog(String name) { super(name); }
    public void makeSound()
    {
        System.out.println(getName() + " barks");
    }
}
```

```
class Cat extends Animal {
    public Cat(String name) { super(name); }
    public void makeSound()
    {
        System.out.println(getName() + " meows");
    }
}

public class
AbstractionExample { public
static void main(String[] args)
    {

        Animal myDog = new
        Dog("Buddy"); Animal myCat =
        new Cat("Fluffy");
        myDog.makeSound();
        myCat.makeSound();
    }
}
```

}

}

Here are some reasons why we use abstract in Java:

1. Abstraction:

- Abstract classes are used to define a generic template for other classes to follow. They define a set of rules and guidelines that their subclasses must follow. By providing an abstract class, we can ensure that the classes that extend it have a consistent structure and behavior. This makes the code more organized and easier to maintain.

2. Polymorphism:

- Abstract classes and methods enable polymorphism in Java. Polymorphism is the ability of an object to take on many forms. This means that a variable of an abstract type can hold objects of any concrete subclass of that abstract class. This makes the code more flexible and adaptable to different situations.

3. Frameworks and APIs:

- Java has numerous frameworks and APIs that use abstract classes. By using abstract classes developers can save time and effort by building on existing code and focusing on the aspects specific to their applications.
- In summary, the abstract keyword is used to provide a generic template for other classes to follow. It is used to enforce consistency, inheritance, polymorphism, and encapsulation in Java.

Advantages of Abstraction

- It reduces the complexity of viewing things. Avoids code duplication and increases reusability.
- Helps to increase the security of an application or program as only essential details are provided to the user.
- It improves the maintainability of the application and the modularity of the application.
- The enhancement will become very easy because without affecting end-users we can able to perform any type of changes in our internal system.
- Improves code reusability and maintainability. Hides implementation details and exposes only relevant information.
- Provides a clear and simple interface to the user. Increases security by preventing access to internal class details.
- Supports modularity, as complex systems can be divided into smaller and more manageable parts.
- Abstraction provides a way to hide the complexity of implementation details from the user, making it easier to understand and use.
- Abstraction allows for flexibility in the implementation of a program, as changes to the underlying implementation details can be made without affecting the user-facing interface.
- Abstraction enables modularity and separation of concerns, making code more maintainable and

easier to debug.

Disadvantages of Abstraction in Java:

- Abstraction can make it more difficult to understand how the system works.
- It can lead to increased complexity, especially if not used properly.
- This may limit the flexibility of the implementation.
- Abstraction can add unnecessary complexity to code if not used appropriately, leading to increased development time and effort.
- Abstraction can make it harder to debug and understand code, particularly for those unfamiliar with the abstraction layers and implementation details.
- Overuse of abstraction can result in decreased performance due to the additional layers of code and indirection.

Abstract Classes

- Java abstract class is a class that can not be initiated by itself, it needs to be subclassed by another class to use its properties. An abstract class is declared using the “abstract” keyword in its class definition.

abstract class Shape

{

```
int color;  
// An abstract function  
abstract void draw();  
}
```

Some important observations about abstract classes are as follows:

- An instance of an abstract class can not be created.
- Constructors are allowed.
- We can have an abstract class without any abstract method.
- There can be a final method in abstract class but any abstract method in class(abstract class) can not be declared as final or in simpler terms final method can not be abstract itself as it will yield an error: “Illegal combination of modifiers: abstract and final”
- We can define static methods in an abstract class
- We can use the abstract keyword for declaring top-level classes (Outer class) as well as inner classes as abstract
- If a class contains at least one abstract method then compulsory should declare a class as abstract
- If the Child class is unable to provide implementation to all abstract methods of the Parent class then we should declare that Child class as abstract so that the next level Child class should provide implementation to the remaining abstract

method

```
abstract class Sunstar {  
    abstract void printInfo();  
}  
// Abstraction performed using  
extends class Employee extends  
Sunstar {  
    void printInfo() {  
        String name =  
            "avinash"; int age = 21;  
        float salary = 222.2F;  
        System.out.println(name);  
        System.out.println(age);  
        System.out.println(salary)  
        ;  
    }  
}  
class Base {  
    public static void main(String args[]) {  
        Sunstar s = new Employee();  
        s.printInfo();  
    }  
}
```


}

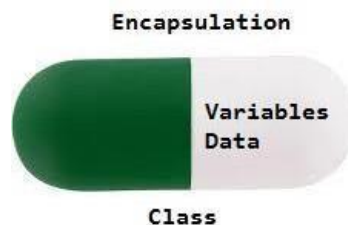
Encapsulation

- Encapsulation is a fundamental concept in object-oriented programming (OOP) that refers to the bundling of data and methods that operate on that data within a single unit, which is called a class in Java.
- Encapsulation is a way of hiding the implementation details of a class from outside access and only exposing a public interface that can be used to interact with the class.
- In Java, encapsulation is achieved by declaring the instance variables of a class as private, which means they can only be accessed within the class. To

allow outside access to the instance variables, public methods called getters and setters are defined, which are used to retrieve and modify the values of the instance variables, respectively.

- By using getters and setters, the class can enforce its own data validation rules and ensure that its internal state remains consistent.

- Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates.
- Another way to think about encapsulation is, that it is a protective shield that prevents the data from being accessed by the code outside this shield.



```
class Person {  
    private String  
    name; private  
    int age;  
    public String getName() { return name; }  
    public void setName(String name) { this.name =  
    name; } public int getAge() { return age; }  
    public void setAge(int age) { this.age = age; }  
}  
  
public class Main {  
    public static void main(String[] args)  
    {  
        Person person = new  
        Person();  
        person.setName("John");  
        person.setAge(30);  
  
        System.out.println("Name: " + person.getName());  
        System.out.println("Age: " + person.getAge());  
    }  
}
```

- Technically in encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of its own class in which it is declared.
- As in encapsulation, the data in a class is hidden from other classes using the data hiding concept which is achieved by making the members or methods of a class private, and the class is exposed to the end-user or the world without providing any details behind implementation using the abstraction concept, so it is also known as a combination of data-hiding and abstraction.
- Encapsulation can be achieved by Declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.
- It is more defined with the setter and getter method.

Advantages of Encapsulation:

1. **Data Hiding:** it is a way of restricting the access of our data members by hiding the implementation details. Encapsulation also provides a way for data hiding. The user will have no idea about the inner implementation of the class. It will not be visible to the user how the class is storing values in the variables. The user will only know that we are passing the values to a setter method and variables are getting initialized with that value.
2. **Increased Flexibility:** We can make the variables of the class read-only or write-only depending on our requirements. If we wish to make the variables read-only then we have to omit the setter methods like setName(), setAge(), etc. from the above program or if we wish to make the variables write-only then we have to omit the get methods like getName(), getAge(), etc. from the above program
3. **Reusability:** Encapsulation also improves the re-usability and is easy to change with new requirements.
4. **Testing code is easy:** Encapsulated code is easy to test for unit testing.
5. **Freedom to programmer in implementing the details of the system:** This is one of the major advantage of encapsulation that it gives the programmer freedom in implementing the details of a system. The only constraint on the programmer is to maintain the abstract interface that outsiders see.

```

class Account {
    // private data members to hide the data private
    long acc_no;
    private String name, email; private
    float amount;
    // public getter and setter methods public long
    getAcc_no() { return acc_no; } public void
    setAcc_no(long acc_no)
    {   this.acc_no = acc_no;   }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; } public
    String getEmail() { return email; }
    public void setEmail(String email)
    {   this.email = email;   }
    public float getAmount() { return amount; } public
    void setAmount(float amount)
    {   this.amount = amount;   }
}

```

```

public class ABC {
    public static void main(String[] args)
    {
        // creating instance of
        Account class Account acc
        = new Account();
        // setting values through setter
        methods
        acc.setAcc_no(7310805450L);
        acc.setName("MD FAIZ");
        acc.setEmail("mdfaiz689@gmai
        l.com");
        acc.setAmount(100000f);
        // getting values through getter
        methods System.out.println(
            acc.getAcc_no() + " " + acc.getName() + " "
            + acc.getEmail() + " " + acc.getAmount());
    }
}

```

String Manipulations

String

- In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string. For example:

```
char[] ch={'h','e','l','l','o','w','o','r','l','d'};
```

```
String s=new String(ch);
```

- is same as:

String s="helloworld";

- Java String class provides a lot of methods to perform operations on strings such as `compare()`, `concat()`, `equals()`, `split()`, `length()`, `replace()`, `compareTo()`, `intern()`, `substring()` etc.

Method	Description	Return Type
charAt()	Returns the character at the specified index (position)	char
compareTo()	Compares two strings lexicographically	int
compareToIgnoreCase()	Compares two strings lexicographically, ignoring case differences	int
concat()	Appends a string to the end of another string	String
contains()	Checks whether a string contains a sequence of characters	boolean
contentEquals()	Checks whether a string contains the exact same sequence of characters of the specified CharSequence or StringBuffer	boolean
copyValueOf()	Returns a String that represents the characters of the character array	String
equals()	Compares two strings. Returns true if the strings are equal, and false if not	boolean
equalsIgnoreCase()	Compares two strings, ignoring case considerations	boolean
getChars()	Copies characters from a string to an array of chars	void
indexOf()	Returns the position of the first found occurrence of specified characters in a string	int

toString()	Returns the value of a String object	String
toUpperCase()	Converts a string to upper case letters	String
trim()	Removes whitespace from both ends of a string	String

Method	Description	Return Type
isEmpty()	Checks whether a string is empty or not	boolean
lastIndexOf()	Returns the position of the last found occurrence of specified characters in a string	int
length()	Returns the length of a specified string	int
matches()	Searches a string for a match against a regular expression, and returns the matches	boolean
replace()	Searches a string for a specified value, and returns a new string where the specified values are replaced	String
replaceFirst()	Replaces the first occurrence of a substring that matches the given regular expression with the given replacement	String
replaceAll()	Replaces each substring of this string that matches the given regular	String

	expression with the given replacement	
split()	Splits a string into an array of substrings	String[]
startsWith()	Checks whether a string starts with specified characters	boolean
toArray()	Converts this string to a new character array	char[]
toLowerCase()	Converts a string to lower case letters	String
valueOf()	Returns the string representation of the specified value	String

String Buffer

- Java StringBuffer class is used to create mutable (modifiable) String objects.
- The StringBuffer class in Java is the same as String class except it is mutable i.e. it can be changed.
- There are several advantages of using StringBuffer over regular String objects in Java:
 1. Mutable: StringBuffer objects are mutable, which means that you can modify the contents of the object after it has been created. In contrast, String objects are

immutable, which means that you cannot change the contents of a String once it has been created.

2. Efficient: Because StringBuffer objects are mutable, they are more efficient than creating new String objects each time you need to modify a string. This is especially true if you need to modify a string multiple times, as each modification to a String object creates a new object and discards the old one.
3. Thread-safe: StringBuffer objects are thread-safe, which means multiple threads cannot access it simultaneously. In contrast, String objects are not thread-safe, which means that you need to use synchronization if you want to access a String object from multiple threads.

- Important Constructors of StringBuffer Class

1. `StringBuffer()`: creates an empty string buffer with an initial capacity of 16.
2. `StringBuffer(String str)`: creates a string buffer with the specified string.
3. `StringBuffer(int capacity)`: creates an empty string buffer with the specified capacity as length.

• Methods of StringBuffer class

Methods	Action Performed
append()	Used to add text at the end of the existing text.
length()	The length of a StringBuffer can be found by the length() method
capacity()	the total allocated capacity can be found by the capacity() method
charAt()	This method returns the char value in this sequence at the specified index.
delete()	Deletes a sequence of characters from the invoking object
deleteCharAt()	Deletes the character at the index specified by <i>loc</i>
ensureCapacity()	Ensures capacity is at least equals to the given minimum.
insert()	Inserts text at the specified index position
length()	Returns length of the string
reverse()	Reverse the characters within a StringBuffer object
replace()	Replace one set of characters with another set inside a StringBuffer object

String Tokenizer

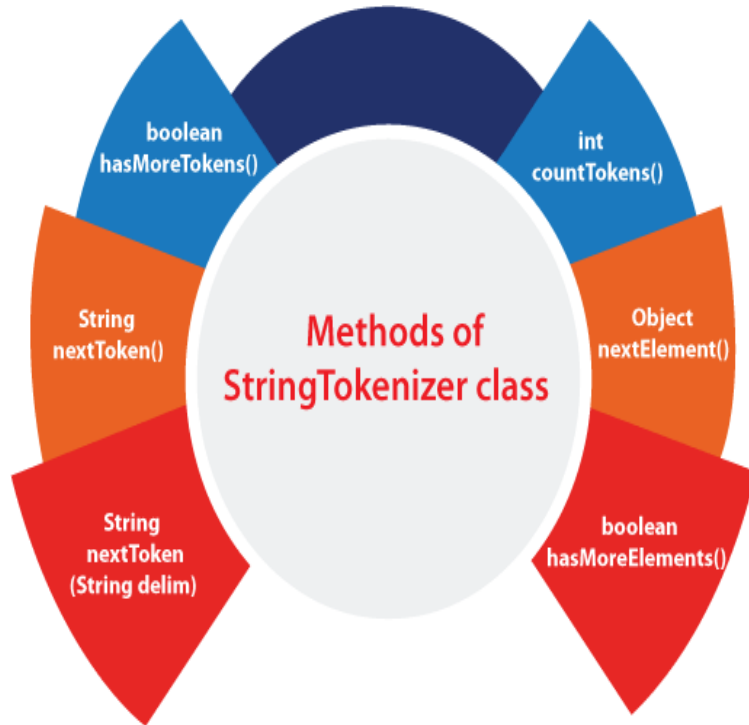
- The `java.util.StringTokenizer` class allows you to break a String into tokens. It is simple way to break a String. It is a legacy class of Java.
- It doesn't provide the facility to differentiate numbers, quoted strings, identifiers etc. like `StreamTokenizer` class. We will discuss about the `StreamTokenizer` class in I/O chapter.

- In the StringTokenizer class, the delimiters can be provided at the time of creation or one by one to the tokens.

- There are 3 constructors defined in the StringTokenizer class.

Constructor	Description
StringTokenizer(String str)	It creates StringTokenizer with specified string.
StringTokenizer(String str, String delim)	It creates StringTokenizer with specified string and delimiter.
StringTokenizer(String str, String delim, boolean returnValue)	It creates StringTokenizer with specified string, delimiter and returnValue. If return value is true, delimiter characters are considered to be tokens. If it is false, delimiter characters serve to separate tokens.

- The six useful methods of the StringTokenizer class are as follows:



Example:

```
import java.util.StringTokenizer;
public class Simple{
    public static void main(String args[]){
        StringTokenizer st = new StringTokenizer("my name is khan"," ");
        while (st.hasMoreTokens()) {
            System.out.println(st.nextToken());
        }
    }
}
```

Output:

my
name
is
khan

Packages

- Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces. Packages are used for:
 1. Preventing naming conflicts. For example there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee
 2. Making searching/locating and usage of classes, interfaces, enumerations and annotations easier
 3. Providing controlled access: protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only.
 4. Packages can be considered as data encapsulation (or data-hiding).
- All we need to do is put related classes into packages. After that, we can simply write an import class from existing packages and use it in our program. A package is a container of a group of related classes where some of the classes are accessible are exposed and others are kept for internal purpose.
- We can reuse existing classes from the packages as many time as we need it in our program.

Introduction to predefined packages

- These packages consist of a large number of classes which are a part of Java API. Some of the commonly used built-in packages are:
 - 1) `java.lang`: Contains language support classes (e.g. `Class` which defines primitive data types, math operations). This package is automatically imported.
 - 2) `java.io`: Contains classes for supporting input / output operations.
 - 3) `java.util`: Contains utility classes which implement data structures like `LinkedList`, `Dictionary` and support ; for Date / Time operations.

- 4) java.applet: Contains classes for creating Applets.
- 5) java.awt: Contain classes for implementing the components for graphical user interfaces (like button , ;menus etc).
- 6) java.net: Contain classes for supporting networking operations.

User Defined Packages

- These are the packages that are defined by the user. First we create a directory myPackage (name should be same as the name of the package). Then create the MyClass inside the directory with the first statement being the package names.

*/*Name of the package must be same as the directory under which this file is saved*/*

package myPackage;

public class MyClass

{

```
public void getNames(String s)  
{  
    System.out.println(s);
```

- Now we can use the MyClass class in our program.

```
/* import 'MyClass' class from 'names' myPackage */
```

```
import  
myPackage.MyClass;  
    }  
}
```

```
public class PrintName  
{  
    public static void main(String args[])  
    {  
        /* Initializing the String variable with a value */  
        String name = "Hello";
```

Core JAVA

UNIT 2

By Shivani Deopa

Exception Handling:

Introduction

- Exception Handling in Java is one of the effective means to handle the runtime errors so that the regular flow of the application can be preserved.
- Java Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException`, `RemoteException`, etc.
- Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions.

- Exceptions can be caught and handled by the program. When an exception occurs within a method, it creates an object. This object is called the exception object.
- It contains information about the exception, such as the name and description of the exception and the state of the program when the exception occurred.

Major reasons why an exception Occurs

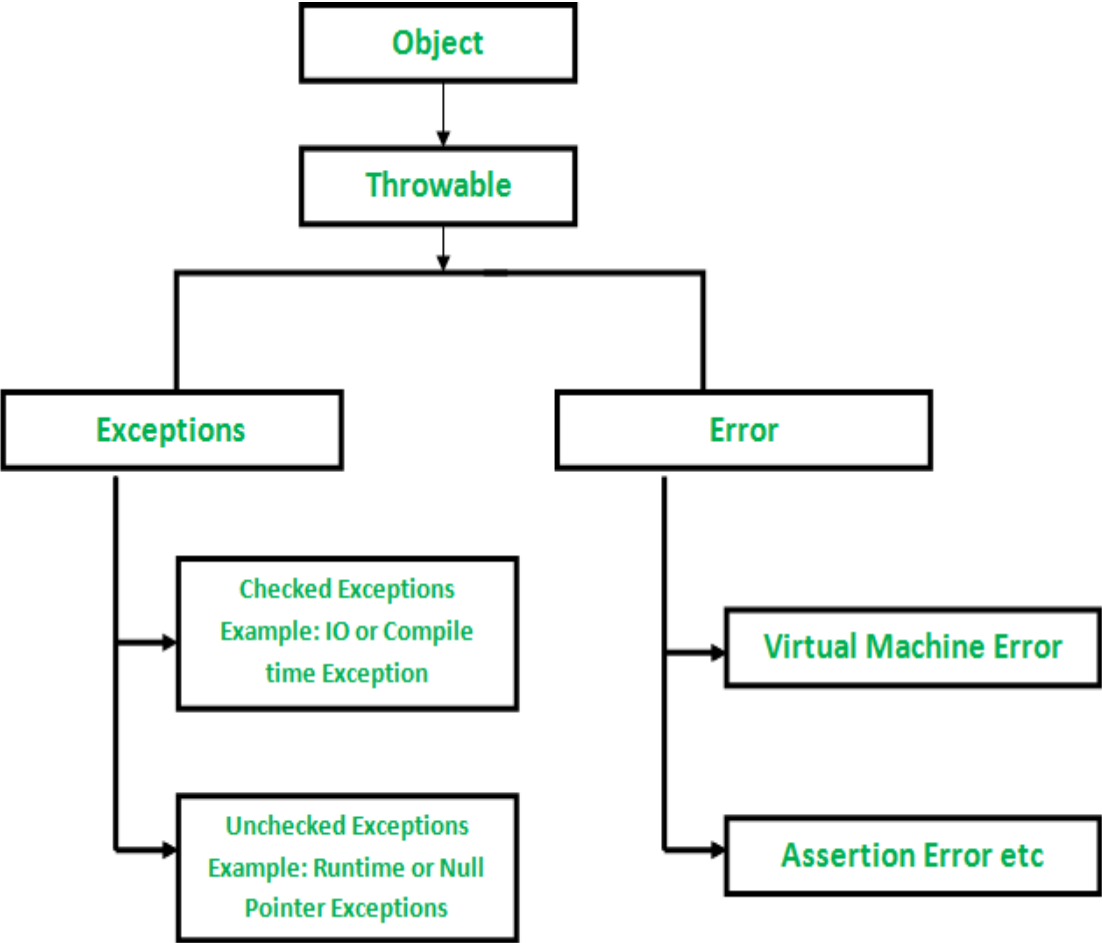
1. Invalid user input
 2. Device failure
 3. Loss of network connection
 4. Physical limitations (out of disk memory)
 5. Code errors
 6. Opening an unavailable file
- Errors represent irrecoverable conditions such as Java virtual machine (JVM) running out of memory, memory leaks, stack overflow errors, library incompatibility, infinite recursion, etc. Errors are usually beyond the control of the programmer, and we should not try to handle errors.

Differences between Error and Exception that is as follows:

- Error: An Error indicates a serious problem that a reasonable application should not try to catch.

- Exception: Exception indicates conditions that a reasonable application might try to catch.

Methods	Description
boolean hasMoreTokens()	It checks if there is more t available.
String nextToken()	It returns the next token f StringTokenizer object.
String nextToken(String delim)	It returns the next token k the delimiter.
boolean hasMoreElements()	It is the same as hasMore method.
Object nextElement()	It is the same as nextToken() but its return type is Object.



```
int countTokens()
```

It returns the total number of tokens.

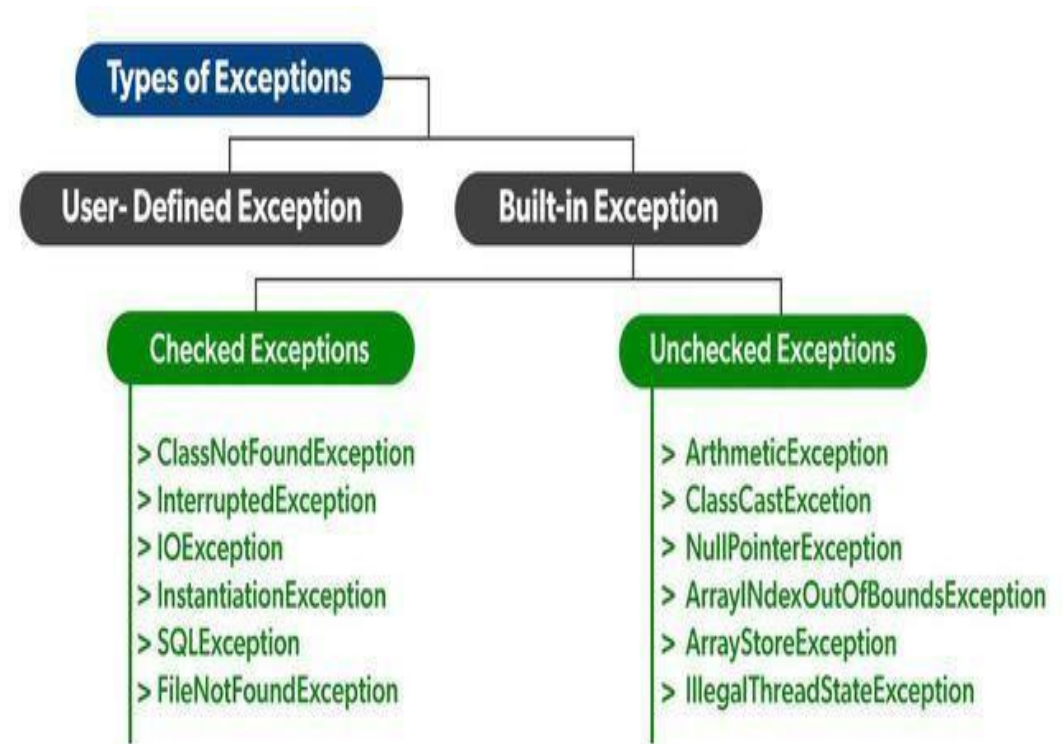
Exception Hierarchy

- All exception and error types are subclasses of class **Throwable**, which is the base class of the hierarchy.
- One branch is headed by **Exception**. This class is used for exceptional conditions that user programs should catch. `NullPointerException` is an example of such an exception.
- Another branch, **Error** is used by the Java run-time system([JVM](#)) to indicate errors having to do with the run-time environment itself(JRE).
- `StackOverflowError` is an example of

such an error.

Types of Exceptions

- Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their own exceptions.
- Exceptions can be categorized in two ways:
 1. Built-in Exceptions



- Checked Exception
- Unchecked Exception

2. User-Defined Exceptions

Pre-Defined Exceptions

- Built-in/Pre-Defined exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations.
- 1. **Checked Exceptions:** Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler.

- 2. Unchecked Exceptions:** The unchecked exceptions are just opposite to the checked exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error.

- There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where ArithmeticException occurs

- If we divide any number by zero, there occurs an ArithmeticException.

int a=50/0;//ArithmeticException

2) A scenario where NullPointerException occurs

- If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

String s=null;

System.out.println(s.length());//NullPointerException

3) A scenario where NumberFormatException occurs

- If the formatting of any variable or number is mismatched, it may result into NumberFormatException. Suppose we have a string variable that has characters; converting this variable into digit will cause NumberFormatException.

```
String s="abc";
```

```
int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where ArrayIndexOutOfBoundsException occurs

- When an array exceeds to it's size, the ArrayIndexOutOfBoundsException occurs. there may be other reasons to occur ArrayIndexOutOfBoundsException. Consider the following statements.

```
int a[]=new int[5];
```

```
a[10]=50; //ArrayIndexOutOfBoundsException
```

Try-Catch-Finally

- Java try block is used to enclose the code that might throw an exception. It must be used within the method.
- If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.
- Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

```
try{  
    //code that may throw an exception  
}catch(Exception_class_Name ref){}
```


Syntax of try-finally block

```
try{
```

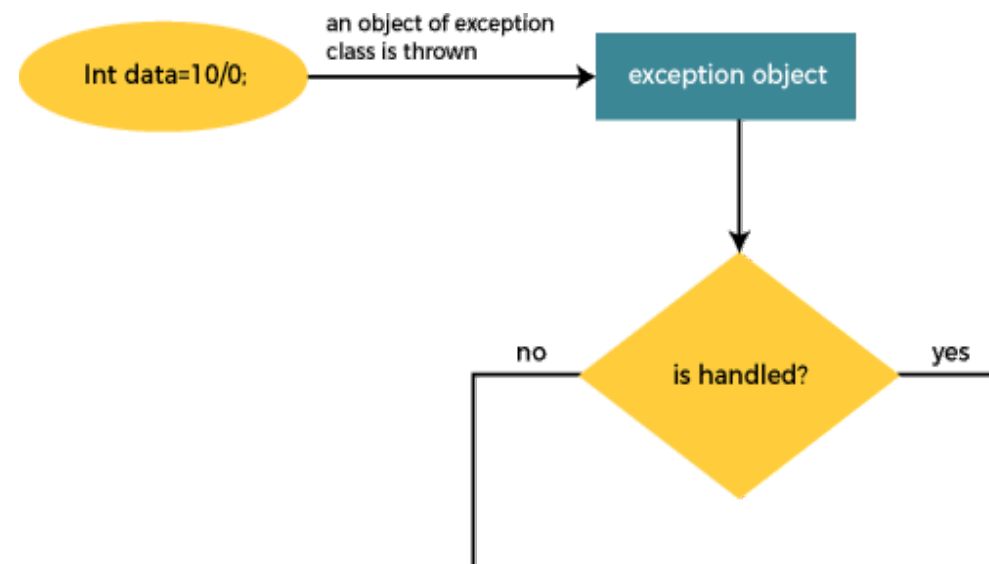
```
//code that may throw an exception
```

```
}finally{}
```

- Java catch block
- Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., Exception) or the generated exception type.
- However, the good approach is to declare the generated type of exception.
- The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Internal Working of Java try-catch block

- The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:
 1. Prints out exception description.
 2. Prints the stack trace (Hierarchy of methods where the exception occurred).



3. Causes the program to terminate.
 - But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed.

- Problem without exception handling
- Let's try to understand the problem if we don't use a try-catch block.

Example 1

```
public class TryCatchExample1 {  
    public static void main(String[] args) {  
        int data=50/0; //may throw  
exception  
        System.out.println("rest of the  
code");  
    }  
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

- Solution by exception handling
- Let's see the solution of the above problem by a java try-catch block.

Example 2

```
public class TryCatchExample2 {  
    public static void main(String[] args) {  
        try  
        {  
  
            int data=50/0; //may throw exception  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Output:

```
java.lang.ArithmeticException: / by zero  
rest of the code
```

Throws

- The Java throws keyword is used to declare an exception. It gives an information to the programmer that there may occur an exception. So, it is better for the programmer to provide the exception handling code so that the normal flow of the program can be maintained.
- Exception Handling is mainly used to handle the checked exceptions. If there occurs any unchecked exception such as NullPointerException, it is programmers' fault that he is not checking the code before it being used.

Syntax of Java throws

```
return_type method_name() throws exception_class_name{  
    //method code  
}
```



```
import java.io.*;
```

```
class M{  
    void method() throws IOException{  
        System.out.println("device operation performed");  
    }  
}
```

```
}  
class Testthrows3{  
    public static void main(String args[])throws IOException{//declare  
exception  
        M m=new M();  
        m.method();  
        System.out.println("normal flow...");  
    }  
}
```

Output:

```
device operation performed  
normal flow
```

}

}

throw

- In Java, exceptions allows us to write good quality codes where the errors are checked at the compile time instead of runtime and we can create custom exceptions making the code recovery and debugging easier.
- The Java throw keyword is used to throw an exception explicitly.
- We specify the exception object which is to be thrown. The Exception has some message with it that provides the error description. These exceptions may be related to user inputs, server, etc.
- We can throw either checked or unchecked exceptions in Java by throw keyword.

It is mainly used to throw a custom exception.

- We can also define our own set of conditions and throw an exception explicitly using throw keyword. For example, we can throw `ArithmeticException` if we divide a number by another number. Here, we just need to set the condition and throw exception using throw keyword.

- The syntax of the Java throw keyword is given below.

throw Instance

- *i.e.,* throw new exception_class("error message");

- Let's see the example of throw IOException.

throw new IOException("sorry device error");

- Where the Instance must be of type Throwable or subclass of Throwable. For example, Exception is the sub class of Throwable and the user-defined exceptions usually extend the Exception class.

```
import java.io.*;

public class TestThrow2 {

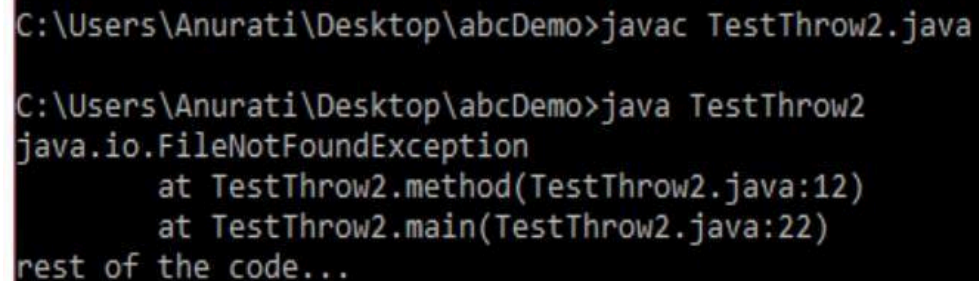
    //function to check if person is eligible to vote or not
    public static void method() throws FileNotFoundException {

        FileReader file = new FileReader("C:\\Users\\Anurati\\Desktop\\abc.txt");
        BufferedReader fileInput = new BufferedReader(file);
        throw new FileNotFoundException();
    }

    public static void main(String args[]){
        try
        { method(); }

        catch (FileNotFoundException e)
        { e.printStackTrace(); }

        System.out.println("rest of the code...");
    }
}
```



```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow2.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow2
java.io.FileNotFoundException
    at TestThrow2.method(TestThrow2.java:12)
    at TestThrow2.main(TestThrow2.java:22)
rest of the code...
```

User Defined Exception examples

- Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called 'user-defined Exceptions'.
- The ***advantages of Exception Handling in Java*** are as follows:

1. Provision to Complete Program Execution
2. Easy Identification of Program Code and Error-Handling Code
3. Propagation of Errors
4. Meaningful Error Reporting
5. Identifying Error Types


```

class InvalidAgeException extends Exception { public
    InvalidAgeException(String str) {
        // calling the constructor of parent Exception
        super(str);
    }
}

public class TestCustomException1 {
    // method to check the age
    static void validate(int age) throws

    InvalidAgeException {

        if (age < 18) {
            // throw an object of user defined exception
            throw new InvalidAgeException("age is not
            valid to vote");
        } else {
            System.out.println("welcome to vote");

```

```

        public static void main(String
            args[]) { try {
                // calling the method
                validate(13);
            } catch (InvalidAgeException ex) {
                System.out.println("Caught the exception");
                System.out.println("Exception occurred: " + ex);
            }

        }
    }
}

```

```
        System.out.println("rest of the code...");  
    }  
}
```

Output:

Caught the exception

*Exception occurred: InvalidAgeException: age is not valid
to vote*

rest of the code...

Multithreading:

- Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU.
- Each part of such program is called a thread. So, threads are light-weight processes within a process.
- Threads can be created by using two mechanisms :
 1. Extending the Thread class
 2. Implementing the Runnable Interface

Thread Creations

- We create a class that extends the `java.lang.Thread` class. This class overrides the `run()` method available in the `Thread` class.
- A thread begins its life inside `run()` method. We create an object of our new class and call `start()` method to start the execution of a thread.
- `Start()` invokes the `run()` method on the `Thread` object.

```

class MultithreadingDemo extends Thread {
    public void run()
    {
        try
            {      System.out.println( "Thread " + Thread.currentThread().getId() + " is running");      }
        catch (Exception e)

```

```

            {      System.out.println("Exception is caught");      }
    }
}

```

Outp

ut

Thread 15 is running

Thread 14 is running

Thread 16 is running

Thread 12 is running

Thread 11 is running

Thread 13 is running

Thread 18 is running

Thread 17 is running

args)

ads

```

    Object = new MultithreadingDemo();

```

Thread creation by implementing the Runnable Interface

- We create a new class which implements `java.lang.Runnable` interface and override `run()` method.
- Then we instantiate a `Thread` object and call `start()` method on this object.

```
class MultithreadingDemo implements Runnable {
    public void run()
    {
        try {System.out.println( "Thread " + Thread.currentThread().getId() + " is running");
        }
        catch (Exception e) {
            System.out.println("Exception is caught");
        }
    }
}

class Multithread {
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i = 0; i < n; i++) {
            Thread object = new Thread(new MultithreadingDemo());
            object.start();
        }
    }
}
```

Output

```
Thread 13 is running
Thread 11 is running
Thread 12 is running
Thread 15 is running
Thread 14 is running
Thread 18 is running
Thread 17 is running
Thread 16 is running
```


Thread Class vs Runnable Interface

- If we extend the Thread class, our class cannot extend any other class because Java doesn't support multiple inheritance. But, if we implement the Runnable interface, our class can still extend other base classes.
- We can achieve basic functionality of a thread by extending Thread class because it provides some inbuilt methods like `yield()`, `interrupt()` etc. that are not available in Runnable interface.
- Using runnable will give you an object that can be shared amongst multiple threads.

Thread Life Cycle

- A thread in Java at any point of time exists in any one of the following states. A thread lies only in one of the shown states at any instant:
 1. New State
 2. Runnable State
 3. Blocked State

4. Waiting State
5. Timed Waiting State
6. Terminated State

Explanation of Different Thread States

- **New:** Whenever a new thread is created, it is always in the new state. For a thread in the new state, the code has not been run yet and thus has not begun its execution.
- **Active:** When a thread invokes the start() method, it moves from the new state to the active state. The active state contains two states within it: one is runnable, and the other is running.
 - 1) **Runnable:** A thread, that is ready to run is then moved to the runnable state. In the runnable state, the thread may be running or may be ready to run at any given instant of time. It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the

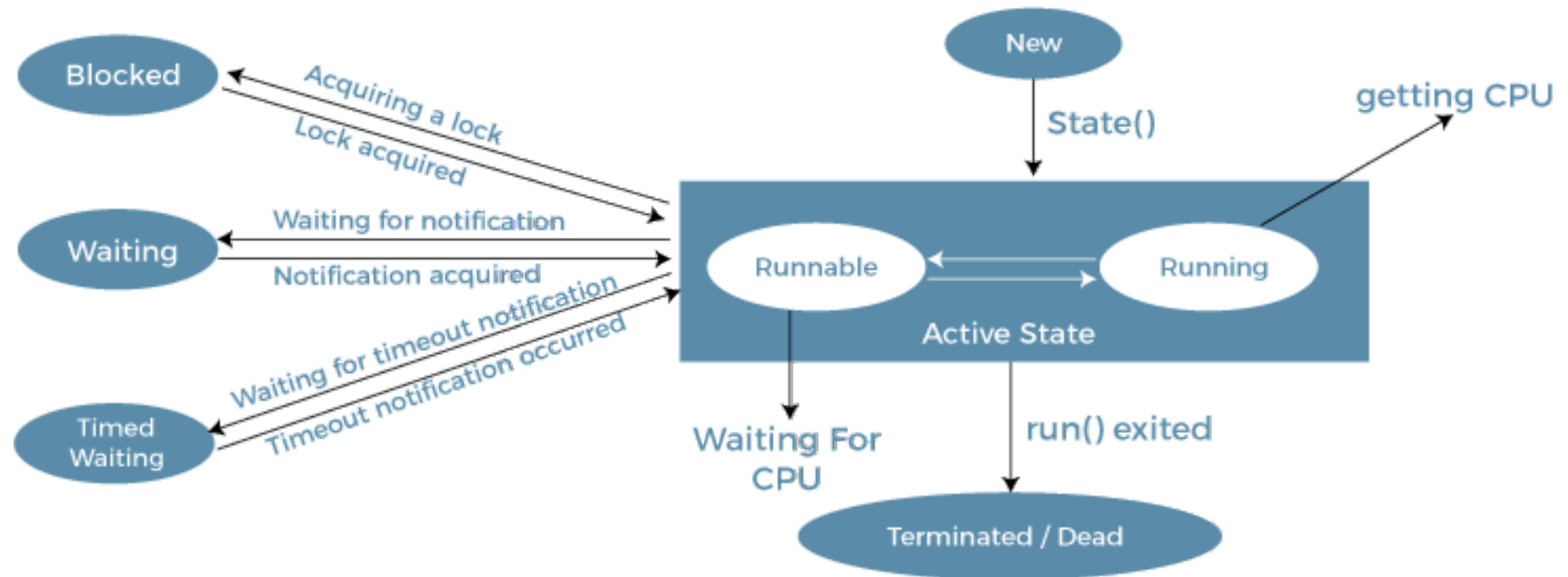
thread the running state.

- A program implementing multithreading acquires a fixed slice of time to each individual thread. Each and every thread runs for a short span of time and when that allocated time slice is over, the thread voluntarily gives up the CPU to the other thread, so that the other threads can also run for their slice of time. Whenever such a scenario occurs, all those threads that are willing to run, waiting for their turn to run, lie in the runnable state. In the runnable state, there is a queue where the threads lie.
- 2) Running: When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.
- 3) Blocked or Waiting: Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state.

- For example, a thread (let's say its name is A) may want to print some data from the printer. However, at the same time, the other thread (let's say its name is B) is using the printer to print some data. Therefore, thread A has to wait for thread B to use the printer. Thus, thread A is in the blocked state. A thread in the blocked state is unable to perform any execution and thus never consume any cycle of the Central Processing Unit (CPU). Hence, we can say that thread A remains idle until the thread scheduler reactivates thread A, which is in the waiting or blocked state.
- When the main thread invokes the `join()` method then, it is said that the main thread is in the waiting state. The main thread then waits for the child threads to complete their tasks. When the child threads complete their job, a notification is sent to the main thread, which again moves the thread from waiting to the active state.
- If there are a lot of threads in the waiting or blocked state, then it is the duty of the thread scheduler to determine which thread to choose and which one to reject, and the chosen thread is then given the opportunity to run.

- **Timed Waiting:** Sometimes, waiting for leads to starvation. For example, a thread (its name is A) has entered the critical section of a code and is not willing to leave that critical section. In such a scenario, another thread (its name is B) has to wait forever, which leads to starvation. To avoid such scenario, a timed waiting state is given to thread B. Thus, thread lies in the waiting state for a specific span of time, and not forever. A real example of timed waiting is when we invoke the sleep() method on a specific thread. The sleep() method puts the thread in the timed wait state. After the time runs out, the thread wakes up and start its execution from when it has left earlier.
- **Terminated:** A thread reaches the termination state because of the following reasons:
 - When a thread has finished its job, then it exists or terminates normally.
 - Abnormal termination: It occurs when some unusual events such as an unhandled exception or segmentation fault.
 - A terminated thread means the thread is no more in the system. In other words, the thread is dead, and there is no way one can respawn (active after kill) the dead thread.

- The following diagram shows the different states involved in the life cycle of a thread.



Life Cycle of a Thread

Life Cycle Methods

1. New

Thread state for a thread that has not yet started.

```
public static final Thread.State NEW
```

2. Runnable

Thread state for a runnable thread. A thread in the runnable state is executing in the Java virtual machine but it may be waiting for other resources from the operating system such as a processor.

public static final Thread.State RUNNABLE

3. Blocked

Thread state for a thread blocked waiting for a monitor lock. A thread in the blocked state is waiting for a monitor lock to enter a synchronized block/method or reenter a synchronized block/method after calling `Object.wait()`.

public static final Thread.State BLOCKED

4. Waiting

Thread state for a waiting thread. A thread is in the waiting state due to calling one of the following methods:

Object.wait with no timeout

Thread.join with no timeout

LockSupport.park

public static final Thread.State WAITING

5. Timed Waiting

Thread state for a waiting thread with a specified waiting time. A thread is in the timed waiting state due to calling one of the following methods with a specified positive waiting time:

Thread.sleep

Object.wait with timeout

Thread.join with timeout

LockSupport.parkNanos

LockSupport.parkUntil

```
public static final Thread.State TIMED_WAITING
```

6. Terminated

Thread state for a terminated thread. The thread has completed execution.

```
public static final Thread.State TERMINATED
```

Synchronization

- Multi-threaded programs may often come to a situation where multiple threads try to access the same resources and finally produce erroneous and unforeseen results.

Why use Java Synchronization?

- Java Synchronization is used to make sure by some synchronization method that only one thread can access the resource at a given point in time.

Java Synchronized Blocks

- Java provides a way of creating threads and synchronizing their tasks using synchronized blocks.
- A synchronized block in Java is synchronized on some object. All synchronized blocks synchronize on the same object and can only have one thread executed inside them at a time.
- All other threads attempting to enter the synchronized block are blocked until the thread inside the synchronized block exits the block.

- There are two synchronizations in Java mentioned below:
 1. Process Synchronization
 2. Thread Synchronization

Process Synchronization in Java

- Process Synchronization is a technique used to coordinate the execution of multiple processes. It ensures that the shared resources are safe and in order.

Thread Synchronization in Java

- Thread Synchronization is used to coordinate and ordering of the execution of the threads in a multi-threaded program. There are two types of thread synchronization are mentioned below:
 1. Mutual Exclusive
 2. Cooperation (Inter-thread communication in Java)

Mutual Exclusive

- Mutual Exclusive helps keep threads from interfering with one another while sharing data. There are three types of Mutual Exclusive mentioned below:
 1. Synchronized method.
 2. Synchronized block.
 3. Static synchronization.

```

class Table{
    synchronized void printTable(int n){//synchronized method
        for(int i=1;i<=5;i++){
            System.out.println(n*i); try{
                Thread.sleep(400);
            }catch(Exception e){System.out.println(e);}
        }
    }
}

class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        t.printTable(5);
    }
}

```

```

}

class MyThread2 extends Thread{
    Table t;
    MyThread2(Table t){ this.t=t;
    }
    public void run(){
        t.printTable(100);
    }
}

public class
TestSynchronization2{ public
    static void main(String args[]){
        Table obj = new Table();//only one
        object MyThread1 t1=new
        MyThread1(obj); MyThread2
        t2=new MyThread2(obj); t1.start();
        t2.start();
    }
}

```

Wait(), notify(), notify all() methods

- The Object class in Java has three final methods that allow threads to communicate about the locked status of a resource.

wait()

It tells the calling thread to give up the lock and go to sleep until some other thread enters the same monitor and calls notify(). The wait() method releases the lock prior to waiting and reacquires the lock prior to returning from the wait() method. The wait() method is actually tightly integrated with the synchronization lock, using a feature not available directly from the synchronization mechanism.

In other words, it is not possible for us to implement the wait() method purely in Java. It is a native method.

General syntax for calling wait() method is like this:

```
synchronized( lockObject )
{
    while( ! condition )
    {
        lockObject.wait();
    }
    //take the action here;
}
```

notify()

It wakes up one single thread that called wait() on the same object. It should be noted that calling notify() does not actually give up a lock on a resource. It tells a waiting thread that that thread can wake up. However, the lock is not actually given up until the notifier's synchronized block has completed.

So, if a notifier calls notify() on a resource but the notifier still needs to perform 10 seconds of actions on the resource within its synchronized block, the thread that had been waiting will need to wait at least another additional 10 seconds for the notifier to release the lock on the object, even though notify() had been called.

General syntax for calling notify() method is like this:

```
synchronized(lockObject)  
{  
    //establish_the_condition;  
    lockObject.notify();  
    //any additional code if needed  
}
```

notifyAll()

It wakes up all the threads that called wait() on the same object. The highest priority thread will run first in most of the situation, though not guaranteed. Other things are same as notify() method above.

General syntax for calling notify() method is like this:

```
synchronized(lockObject)  
{  
    establish_the_condition;
```

```
lockObject.notifyAll();  
}
```


Networking:

Introduction

- When computing devices such as laptops, desktops, servers, smartphones, and tablets and an eternally-expanding arrangement of IoT gadgets such as cameras, door locks, doorbells, refrigerators, audio/visual systems, thermostats, and various sensors are sharing information and data with each other is known as networking.
- In simple words, the term network programming or networking

associates with writing programs that can be executed over various computer devices, in which all the devices are connected to each other to share resources using a network. Here, we are going to discuss ***Java Networking***.

What is Java Networking?

- Networking supplements a lot of power to simple programs. With networks, a single program can regain information stored in millions of computers positioned anywhere in the world.
- Java is the leading programming language composed from scratch with networking in mind. Java Networking is a notion of combining two or more computing devices together to share resources.
- All the Java program communications over the network are done at the application layer.
- The `java.net` package of the J2SE APIs comprises various classes and interfaces that execute the low-level communication features, enabling the user to formulate programs that focus on resolving the problem.

Common Network Protocols

- As stated earlier, the java.net package of the Java programming language includes various classes and interfaces that provide an easy-to-use means to access network resources. Other than classes and interfaces, the java.net package also provides support for the two well-known network protocols. These are:
 1. **Transmission Control Protocol (TCP)** – TCP or Transmission Control Protocol allows secure communication between different applications. TCP is a connection-oriented protocol which means that once a connection is established, data can be transmitted in two directions. This protocol is typically used over the Internet Protocol. Therefore, TCP is also referred to as TCP/IP. TCP has built-in methods to examine for errors and ensure the delivery of data in the order it was sent, making it a complete protocol for transporting information like still images, data files, and web pages.
 2. **User Datagram Protocol (UDP)** – UDP or User Datagram Protocol is a connection-less protocol that allows data packets to be transmitted between different applications. UDP is a simpler Internet protocol in which error-checking and recovery services are not required. In UDP, there is no overhead for opening a connection, maintaining a connection, or terminating a connection. In UDP, the data is continuously sent to the recipient, whether they receive it or not.

Java Networking Terminology

- In Java Networking, many terminologies are used frequently. These widely used Java Networking Terminologies are given as follows:
- **IP Address** – An IP address is a unique address that distinguishes a device on

the internet or a local network. IP stands for “Internet Protocol.” It comprises a set of rules governing the format of data sent via the internet or local network. IP Address is referred to as a logical address that can be modified. It is composed of octets. The range of each octet varies from 0 to 255.

Range of the IP Address – 0.0.0.0 to 255.255.255.255

For Example – 192.168.0.1

- **Port Number** – A port number is a method to recognize a particular process connecting internet or other network information when it reaches a server. The port number is used to identify different applications uniquely. The port number behaves as a communication endpoint among applications. The port number is correlated with the IP address for transmission and communication among two applications. There are 65,535 port numbers, but not all are used every day.

- **Protocol** – A network protocol is an organized set of commands that define how data is transmitted between different devices in the same network. Network protocols are the reason through which a user can easily communicate with people all over the world and thus play a critical role in modern digital communications. For Example – TCP, FTP, POP, etc.
- **MAC Address** – MAC address stands for Media Access Control address. It is a bizarre identifier that is allocated to a NIC (Network Interface Controller/ Card). It contains a 48 bit or 64-bit address, which is combined with the network adapter. MAC address can be in hexadecimal composition. In simple words, a MAC address is a unique number that is used to track a device in a network.
- **Socket** – A socket is one endpoint of a two-way communication connection between the two applications running on the network. The socket mechanism presents a method of inter-process communication (IPC) by setting named contact points between which the communication occurs. A socket is tied to a port number so that the TCP layer can recognize the application to which the data is intended to be sent.
- **Connection-oriented and connection-less protocol** – In a connection-oriented service, the user must establish a connection before starting the communication. When the connection is established, the user can send the message or the information, and after this, they can release the connection. However, In connectionless protocol, the data is transported in one route from source to destination without verifying that the destination is still there or not or if it is ready to receive the message. Authentication is not needed in the connectionless protocol.

Example of Connection-oriented Protocol – Transmission Control Protocol (TCP)

Example of Connectionless Protocol – User Datagram Protocol (UDP)

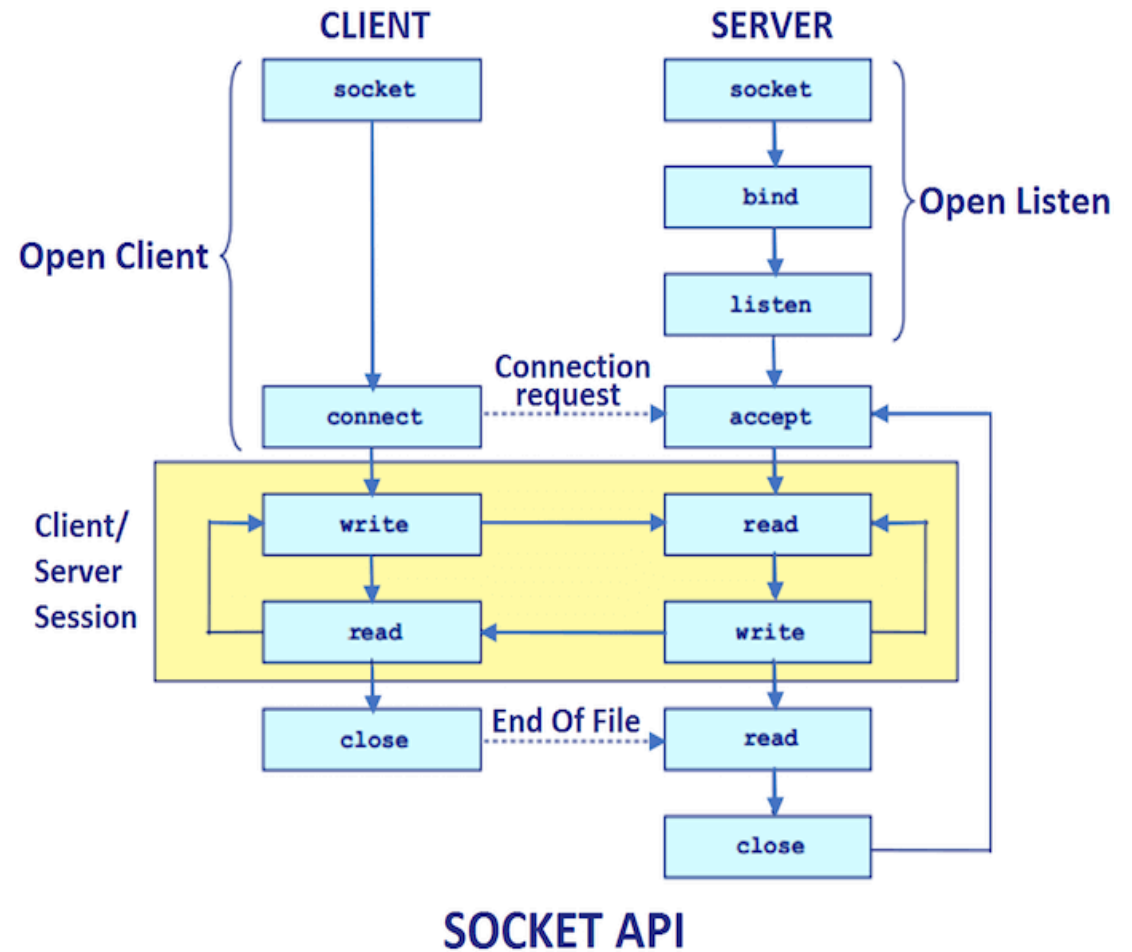
Socket

- Java Socket programming is used for communication between the applications running on different JRE.
- Java Socket programming can be connection-oriented or connection-less.
- Socket and ServerSocket classes are used for connection-oriented socket programming and DatagramSocket and DatagramPacket

classes are used for connection-less socket programming.

- The client in socket programming must know two information:
 1. IP Address of Server, and
 2. Port number.

- Here, we are going to make one-way client and server communication. In this application, client sends a message to the server, server reads the message and prints it. Here, two classes are being used: Socket and ServerSocket.
- The Socket class is used to communicate client and server. Through this class, we can read and write message. The ServerSocket class is used at server-side.
- The accept() method of ServerSocket class blocks the console until the client is connected.
- After the successful connection of client, it returns the instance of Socket at server-side.



- A socket is simply an endpoint for communications between the machines. The Socket class can be used to create a socket.
- Important methods

Method	Description
1) public InputStream getInputStream()	returns the InputStream attached with this socket.
2) public OutputStream getOutputStream()	returns the OutputStream attached with this socket.
3) public synchronized void close()	closes this socket

Server socket

- The ServerSocket class can be used to create a server socket. This object is used to establish communication with the clients.
- Important methods

Method	Description
--------	-------------

1) public Socket accept()	returns the socket and establish a connection between server and client.
2) public synchronized void close()	closes the server socket.

Creating Server:

- To create the server application, we need to create the instance of ServerSocket class. Here, we are using 6666 port number for the communication between the client and server. You may also choose any other port number. The accept() method waits for the client. If clients connects with the given port number, it returns an instance of Socket.

ServerSocket ss=new ServerSocket(6666);

Socket s=ss.accept();//establishes connection and waits for the client

Creating Client:

- To create the client application, we need to create the instance of Socket class. Here, we need to pass the IP address or hostname of the Server and a port number. Here, we are using "localhost" because our server is running on same system.

Socket s=new Socket("localhost",6666);

Client –Server Communication

```
import java.net.*; class
MyServer{
public static void main(String args[])throws Exception{
ServerSocket ss=new ServerSocket(3333);
Socket s=ss.accept();
DataInputStream din=new DataInputStream(s.getInputStream());
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
BufferedReader br=new BufferedReader(new
InputStreamReader(System.in));
String str="",str2=""; while(!str.equals("stop")){
str=din.readUTF(); System.out.println("client
says: "+str); str2=br.readLine();
dout.writeUTF(str2);
dout.flush();  }
din.close();
s.close(); ss.close();
}}
```

```
import
java.net.*;
class
MyClient
{
    public static void main(String args[])throws
    Exception{
        Socket s=new
        Socket("localhost",3333);
        DataInputStream din=new DataInputStream(s.getInputStream());
        DataOutputStream dout=new DataOutputStream(s.getOutputStream());

        BufferedReader br=new
        BufferedReader(new
        InputStreamReader(System.in));

        String
        str="",str2="";

        while(!str.equals
        ("stop")){
            str=br.readLine();
            dout.writeUTF(st
            r);
            dout.flush();
            str2=din.readUTF
            ();
```

```
        System.out.println("Server says: "+str2);
        }
        dout.close();
        s.close();
    }
}
```


RMI

- The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java.
- The RMI allows an object to invoke methods on an object running in another JVM.
- The RMI provides remote communication between the applications

using two objects stub and skeleton.

Understanding stub and skeleton

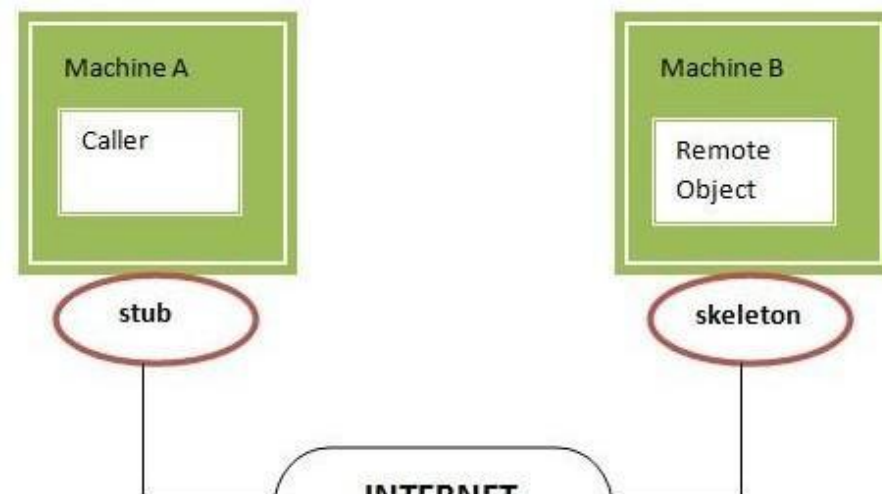
- RMI uses stub and skeleton object for communication with the remote object.
- A remote object is an object whose method can be invoked from another JVM.
Let's understand the stub and skeleton objects:

stub

- The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:
 1. It initiates a connection with remote Virtual Machine (JVM),
 2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
 3. It waits for the result
 4. It reads (unmarshals) the return value or exception, and
 5. It finally, returns the value to the caller.

skeleton

- The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:
 1. It reads the parameter for the remote method
 2. It invokes the method on the actual remote object, and
 3. It writes and transmits (marshals) the result to the caller.

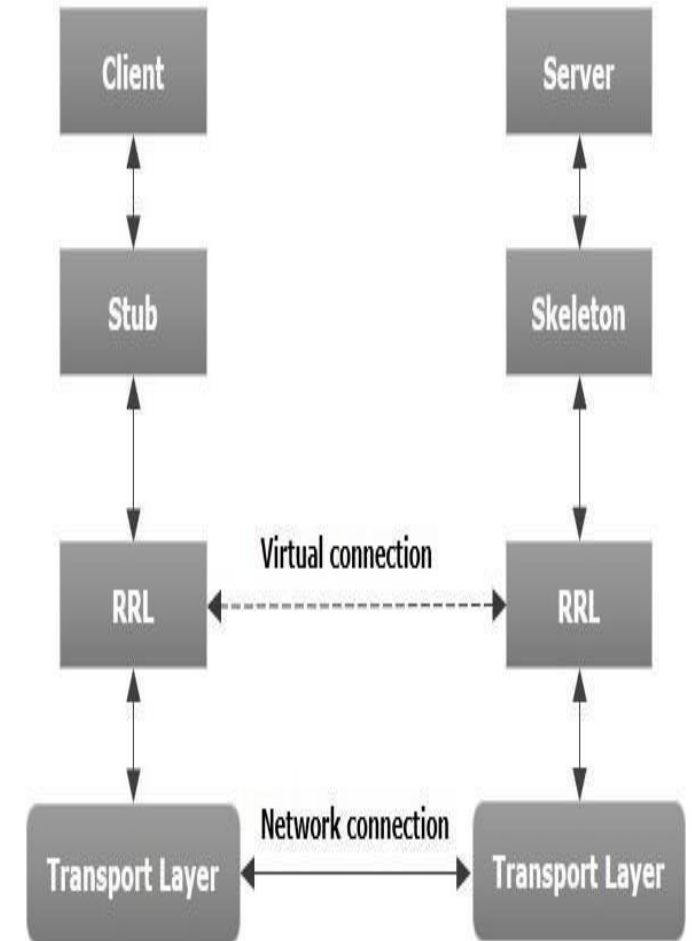


Architecture of an RMI Application

- In an RMI application, we write two programs, a server program (resides on the server) and a client program (resides on the client).
- Inside the server program, a remote object is created and reference of that object is made available for the client (using the registry).
- The client program requests the remote objects on the server and tries to invoke its methods.

The components of this architecture.

1. **Transport Layer** – This layer connects the client and the server. It manages the existing connection and also sets up new connections.
2. **Stub** – A stub is a representation (proxy) of the remote object at client. It resides in the client system; it acts as a gateway for the client program.
3. **Skeleton** – This is the object which resides on the server side. stub communicates with this skeleton to pass request to the remote object.
4. **RRL(Remote Reference Layer)** – It is the layer which manages the references made by the client to the remote object.



Working of an RMI Application

- When the client makes a call to the remote object, it is received by the stub which eventually passes this request to the RRL.
- When the client-side RRL receives the request, it invokes a method

called **invoke()** of the object **remoteRef**. It passes the request to the RRL on the server side.

- The RRL on the server side passes the request to the Skeleton (proxy on the server) which finally invokes the required object on the server.
- The result is passed all the way back to the client.

Goals of RMI

1. To minimize the complexity of the application.
2. To preserve type safety.

3. Distributed garbage collection.
4. Minimize the difference between working with local and remote objects.

Core JAVA

UNIT 3

By Shivani Deopa

AWT:

Introduction

- Java AWT (Abstract Window Toolkit) is an API to develop Graphical User Interface (GUI) or windows-based applications in Java.
- Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system.
- AWT is heavy weight i.e. its components are using the resources

of underlying operating system (OS).

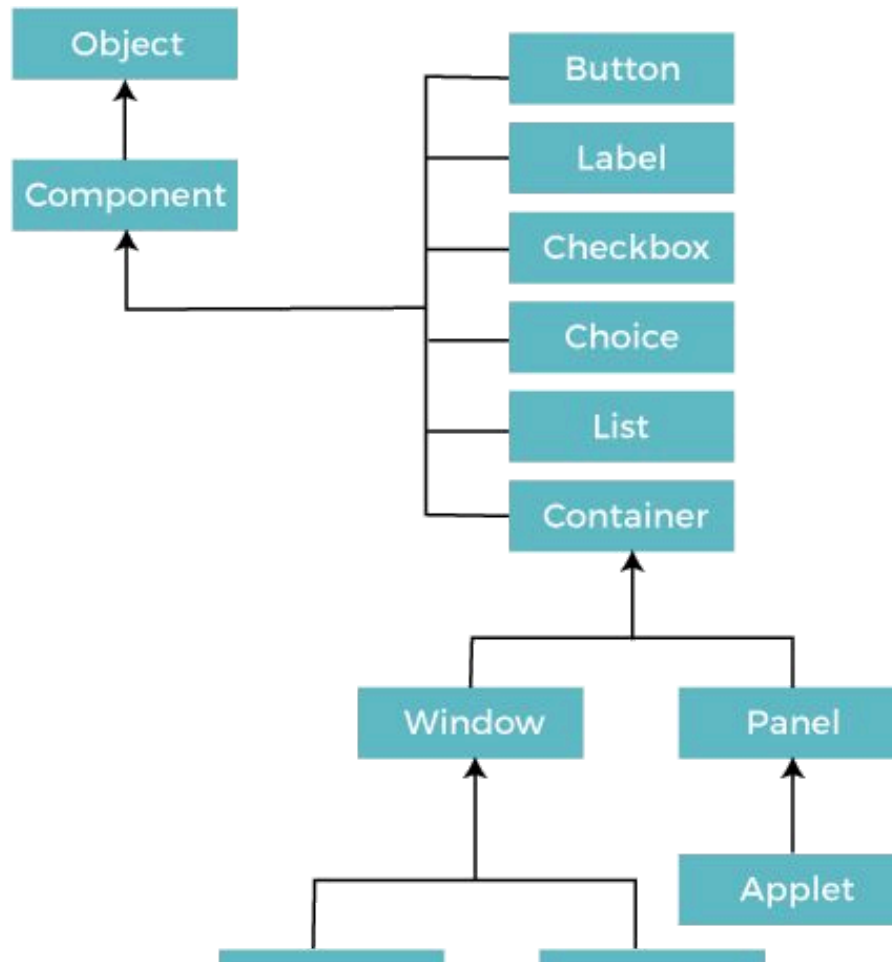
- The java.awt package provides classes for AWT API such as TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.

Why AWT is platform independent?

- Java AWT calls the native platform (operating systems) subroutine for creating API components like TextField, ChechBox, button, etc.
- For example, an AWT GUI with components like TextField, label and button will have different look and feel for the different platforms like Windows, MAC OS, and Unix.
- The reason for this is the platforms have different view for their native components and AWT directly calls the native subroutine that creates those components.
- In simple words, an AWT application will look like a windows application in Windows OS whereas it will look like a Mac application in the MAC OS.

Java AWT Hierarchy

- The hierarchy of Java AWT classes are given below.



Components

Components

- All the elements like the button, text fields, scroll bars, etc. are called components. In Java AWT, there are classes for each component as shown in above diagram. In order to place every component in a particular position on a screen, we need to add them to a container.

Container

- The Container is a component in AWT that can contain another

components like buttons, textfields, labels etc. The classes that extends Container class are known as container such as Frame, Dialog and Panel.

- It is basically a screen where the where the components are placed at their specific locations. Thus it contains and controls the layout of components.

Types of containers:

- There are four types of containers in Java AWT:
- Window
- Panel
- Frame
- Dialog

1. **Window**

- The window is the container that have no borders and menu bars. You must use frame, dialog or another window for creating a window. We need to create an instance of Window class to create this container.

2. **Panel**

- The Panel is the container that doesn't contain title bar, border or menu bar. It is generic container for holding the components. It can have other components like button, text field etc. An instance of Panel class creates a container, in which we can add components.

3. **Frame**

- The Frame is the container that contain title bar and border and can have menu bars. It can have other components like button, text field, scrollbar etc. Frame is most widely used container while developing an AWT application.

Event-Delegation-Model

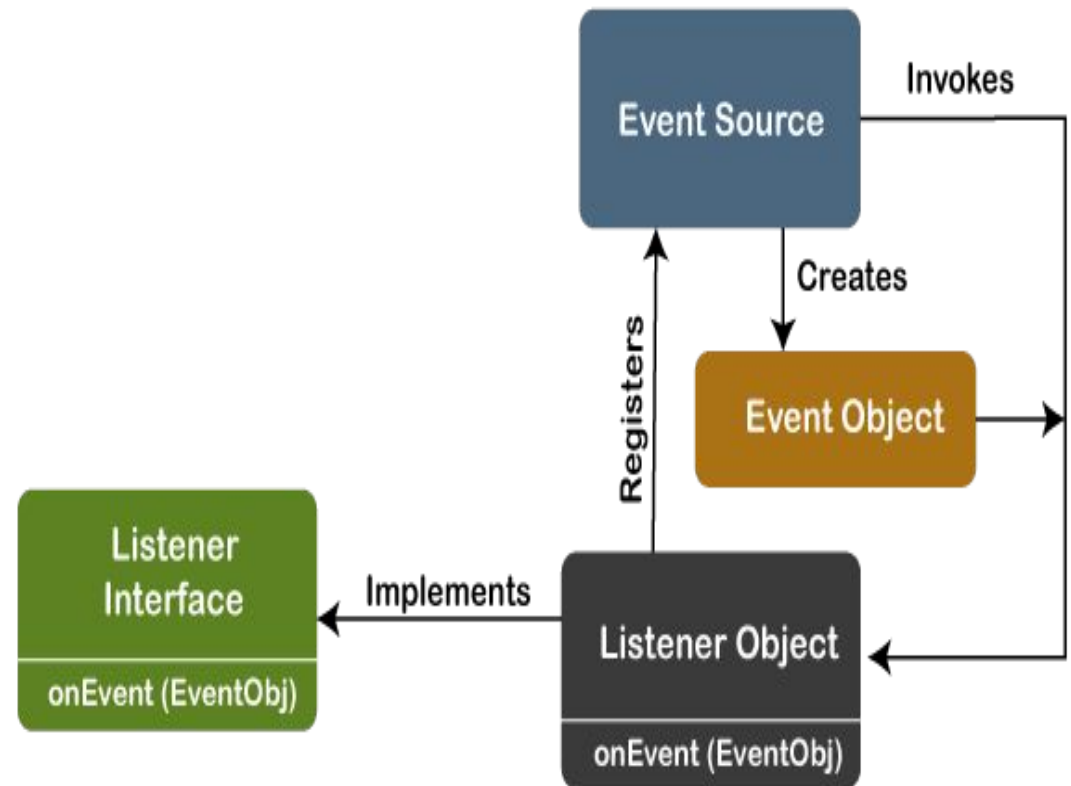
- The Delegation Event model is defined to handle events in GUI programming languages.
- The GUI stands for Graphical User Interface, where a user graphically/visually interacts with the system.
- The GUI programming is inherently event-driven; whenever a user initiates an activity such as a mouse activity, clicks, scrolling, etc., each

is known as an event that is mapped to a code to respond to functionality to the user.

- This is known as event handling.

Event Processing in Java

- Java support event processing since Java 1.0. It provides support for AWT (Abstract Window Toolkit), which is an API used to develop the Desktop application.
- In Java 1.0, the AWT was based on inheritance. To catch and process GUI events for a program, it should hold subclass GUI components and override action() or handleEvent() methods.



- The image demonstrates the event processing.

- But, the modern approach for event processing is based on the Delegation Model. It defines a standard and compatible mechanism to generate and process events. In this model, a source generates an event and forwards it to one or more listeners.
- The listener waits until it receives an event. Once it receives the event, it is processed by the listener and returns it. The UI elements are able to delegate the processing of an event to a separate function.
- The key advantage of the Delegation Event Model is that the application logic is completely separated from the interface logic.
- In this model, the listener must be connected with a source to receive the event notifications. Thus, the events will only be received by the listeners who wish to receive them. So, this approach is more convenient than the inheritance-based event model (in Java 1.0).

- In the older model, an event was propagated up the containment until a component was handled. This needed components to receive events that were not processed, and it took lots of time. The Delegation Event model overcame this issue.
- Basically, an Event Model is based on the following three components:
 1. Events
 2. Events Sources
 3. Events Listeners

Events

- The Events are the objects that define state change in a source. An event can be generated as a reaction of a user while interacting with GUI elements. Some of the event generation activities are moving the mouse pointer, clicking on a button, pressing the keyboard key, selecting an item from the list, and so on. We can also consider many other user operations as events.
- The Events may also occur that may be not related to user interaction, such as a timer expires, counter exceeded, system failures, or a task is completed, etc. We can define events for any of the applied actions.

Event Sources

- A source is an object that causes and generates an event. It generates an event when the internal state of the object is changed. The sources are allowed to generate several different types of events.
- A source must register a listener to receive notifications for a specific event. Each event contains its registration method. Below is an example:

public void addTypeListener (TypeListener e1)

- From the above syntax, the Type is the name of the event, and e1 is a reference to the event listener. For example, for a keyboard event listener, the method will be called as addKeyListener().
- For the mouse event listener, the method will be called as addMouseMotionListener(). When an event is triggered using the respected source, all the events will be notified to registered listeners and receive the event object.
- This process is known as event multicasting. In few cases, the event notification will only be sent to listeners that register to receive

them.

- Some listeners allow only one listener to register. Below is an example:

*public void addTypeListener(TypeListener e2) throws
java.util.TooManyListenersException*

- From the above syntax, the Type is the name of the event, and e2 is the event listener's reference. When the specified event occurs, it will be notified to the registered listener. This process is known as unicasting events.
- A source should contain a method that unregisters a specific type of event from the listener if not needed. Below is an example of the method that will remove the event from the listener.

public void removeTypeListener(TypeListener e2?)

- From the above syntax, the Type is an event name, and e2 is the reference of the listener. For example, to remove the keyboard listener, the removeKeyListener() method will be called.
- The source provides the methods to add or remove listeners that generate the events. For example, the Component class contains the methods to operate on the different types of events, such as adding or removing them from the listener.

Event Listeners

- An event listener is an object that is invoked when an event triggers. The listeners require two things; first, it must be registered with a source; however, it can be registered with several resources to receive notification about the events. Second, it must implement the methods to receive and process the received notifications.
- The methods that deal with the events are defined in a set of interfaces. These interfaces can be found in the `java.awt.event` package.
- For example, the `MouseMotionListener` interface provides two methods when the mouse is dragged and moved. Any object can receive and process these events if it implements the `MouseMotionListener` interface.

Types of Events

- The events are categorized into the following two categories:

1. The Foreground Events:

- The foreground events are those events that require direct interaction of the user. These types of events are generated as a result of user interaction with the GUI component. For example, clicking on a button, mouse movement, pressing a keyboard key, selecting an option from the list, etc.

2. The Background Events :

- The Background events are those events that result from the interaction of the end-user. For example, an Operating system interrupts system failure (Hardware or Software).

- To handle these events, we need an event handling mechanism that provides control over the events and responses.

The Delegation Model

- The Delegation Model is available in Java since Java 1.1. it provides a new delegation-based event model using AWT to resolve the event problems. It provides a convenient mechanism to support complex Java programs.

Design Goals

- The design goals of the event delegation model are as following:
- It is easy to learn and implement
- It supports a clean separation between application and GUI code.
- It provides robust event handling program code which is less error-prone (strong compile-time checking)
- It is Flexible, can enable different types of application models for event flow and propagation.
- It enables run-time discovery of both the component-generated events as well as observable events.
- It provides support for the backward binary compatibility with the previous model.

Event and Listener (Java Event Handling)

- Changing the state of an object is known as an event. For example, click on button, dragging mouse etc.
- The `java.awt.event` package provides many event classes and Listener interfaces for event handling.

Java Event classes and Listener interfaces

Registration Methods

- For registering the component with the Listener, many classes provide the registration methods. For example:

- 1) Button: `public void addActionListener(ActionListener a){}`
- 2) MenuItem: `public void addActionListener(ActionListener a){}`
- 3) TextField: `public void addActionListener(ActionListener a){}`
 `public void addTextListener(TextListener a){}`
- 4) TextArea: `public void addTextListener(TextListener a){}`
- 5) Checkbox: `public void addItemListener(ItemListener a){}`
- 6) Choice: `public void addItemListener(ItemListener a){}`
- 7) List: `public void`
 `addActionListener(ActionListener a){}` `public void`
 `addItemListener(ItemListener a){}`

Layouts

- The LayoutManagers are used to arrange components in a particular manner. The Java LayoutManagers facilitates us to control the positioning and size of the components in GUI forms. LayoutManager is an interface that is implemented by all the classes of layout managers. There are the following classes that represent the layout managers:

1. `java.awt.BorderLayout`
2. `java.awt.FlowLayout`
3. `java.awt.GridLayout`
4. `java.awt.CardLayout`

5. `java.awt.GridBagLayout`
6. `javax.swing.BoxLayout`
7. `javax.swing.GroupLayout`
8. `javax.swing.ScrollPaneLayout`
9. `javax.swing.SpringLayout` etc.

Java BorderLayout

- The BorderLayout is used to arrange the components in five regions: north, south, east, west, and center. Each region (area) may contain one component only. It is the default layout of a frame or window. The BorderLayout provides five constants for each region:

public static final int NORTH

public static final int SOUTH

public static final int EAST

public static final int WEST

public static final int CENTER

Constructors of BorderLayout class:

1. BorderLayout(): creates a border layout but with no gaps between the components.
2. BorderLayout(int hgap, int vgap): creates a border layout with the given horizontal and vertical gaps between the components.

- Example of BorderLayout class: Using BorderLayout() constructor

```
import java.awt.*; import
javax.swing.*; public class
Border
{

JFrame f;
Border()
{

    f = new JFrame();
    JButton b1 = new JButton("NORTH");;
    JButton b2 = new JButton("SOUTH");;
    JButton b3 = new JButton("EAST");; JButton
    b4 = new JButton("WEST");; JButton b5 =
    new JButton("CENTER");;
```

```
f.add(b1,
BorderLayout.NORTH);
f.add(b2,
BorderLayout.SOUTH);
f.add(b3,
BorderLayout.EAST);
f.add(b4,
BorderLayout.WEST);
f.add(b5,
BorderLayout.CENTER);
f.setSize(300, 300);
f.setVisible(true); }
public static void main(String[] args) {
```

Event Classes	Listener Interfaces
---------------	---------------------

ActionEvent	ActionListener
MouseEvent	MouseListener and MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

new Border();

}

}

Java GridLayout

- The Java GridLayout class is used to arrange the components in a rectangular grid. One component is displayed in each rectangle.

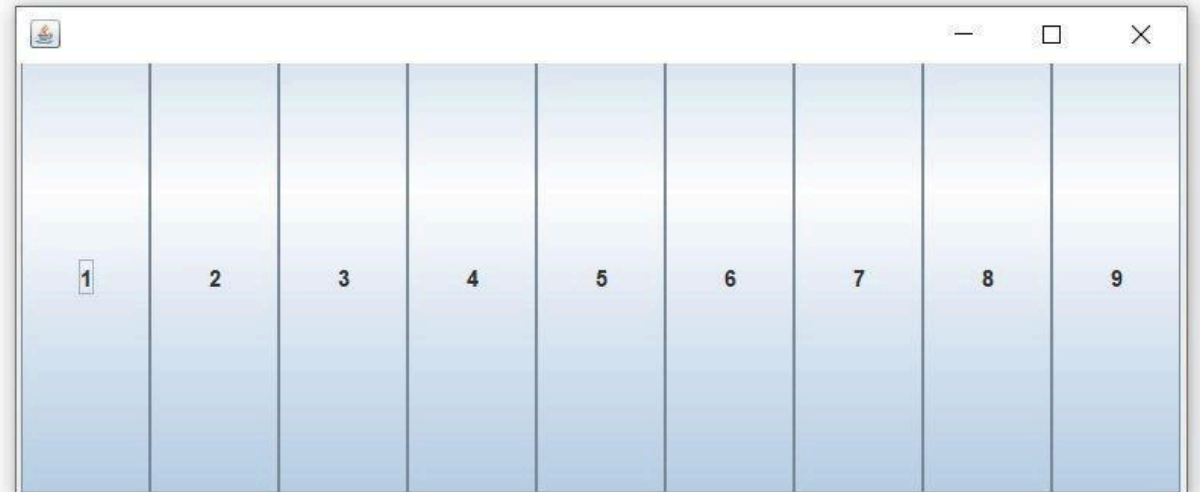
Constructors of GridLayout class

1. GridLayout(): creates a grid layout with one column per component in a row.
2. GridLayout(int rows, int columns): creates a grid layout with the given rows and columns but no gaps between the components.
3. GridLayout(int rows, int columns, int hgap, int vgap): creates a grid layout with the given rows and columns along with given horizontal and vertical gaps.

- Example of GridLayout class: Using GridLayout() Constructor
- The GridLayout() constructor creates only one row. The following example shows the usage of the parameterless constructor.

```
import java.awt.*;  
import javax.swing.*;  
public class GridLayoutExample  
{  
    JFrame frameObj;  
    GridLayoutExample()  
    {  
        frameObj = new JFrame(); JButton  
        btn1 = new JButton("1"); JButton  
        btn2 = new JButton("2"); JButton  
        btn3 = new JButton("3"); JButton  
        btn4 = new JButton("4"); JButton  
        btn5 = new JButton("5"); JButton  
        btn6 = new JButton("6"); JButton  
        btn7 = new JButton("7"); JButton  
        btn8 = new JButton("8"); JButton  
        btn9 = new JButton("9");
```

```
        frameObj.add(btn1); frameObj.add(btn2); frameObj.add(btn3)  
        ;  
        frameObj.add(btn4); frameObj.add(btn5); frameObj.add(btn6)  
        ;  
        frameObj.add(btn7); frameObj.add(btn8); frameObj.add(btn9)  
        ;  
        // setting the grid layout using the parameterless constructor  
  
        frameObj.setLayout(new  
        GridLayout());  
        frameObj.setSize(300, 300);
```



- Example of GridLayout class: Using GridLayout(int rows, int columns) Constructor

```
import java.awt.*; import
javax.swing.*;
public class MyGridLayout{
JFrame f;
MyGridLayout(){ f=new
    JFrame();
    JButton  b1=new    JButton("1");
    JButton  b2=new    JButton("2");
    JButton  b3=new    JButton("3");
    JButton  b4=new    JButton("4");
    JButton  b5=new    JButton("5");
    JButton  b6=new    JButton("6");
    JButton  b7=new    JButton("7");
    JButton  b8=new    JButton("8");
    JButton b9=new JButton("9");
```

```
// adding buttons to the
    frame f.add(b1); f.add(b2);
    f.add(b3);
    f.add(b4); f.add(b5); f.add(b6);

    f.add(b7); f.add(b8); f.add(b9);
    // setting grid layout of 3 rows and 3 columns
    f.setLayout(new GridLayout(3,3));
    f.setSize(300,300);
    f.setVisible(true);

}
public static void main(String[]
    args) { new MyGridLayout();
            }
}
```

Java FlowLayout

- The Java FlowLayout class is used to arrange the components in a line, one after another (in a flow). It is the default layout of the applet or panel.

- Fields of FlowLayout class

public static final int LEFT

public static final int RIGHT

public static final int CENTER

public static final int LEADING

public static final int TRAILING

Constructors of FlowLayout class

1. FlowLayout(): creates a flow layout with centered alignment and a default 5 unit horizontal and vertical gap.
2. FlowLayout(int align): creates a flow layout with the given alignment and a default 5 unit horizontal and vertical gap.
3. FlowLayout(int align, int hgap, int vgap): creates a flow layout with the given alignment and the given horizontal and vertical gap.

- Example of FlowLayout class: Using FlowLayout() constructor

```
import java.awt.*; import
javax.swing.*;
public class FlowLayoutExample
{
JFrame frameObj;
FlowLayoutExample()
{
    frameObj = new JFrame(); JButton b1 = new
    JButton("1"); JButton b2 = new
    JButton("2"); JButton b3 = new
    JButton("3"); JButton b4 = new
    JButton("4"); JButton b5 = new
    JButton("5"); JButton b6 = new
    JButton("6"); JButton b7 = new
    JButton("7"); JButton b8 = new
    JButton("8"); JButton b9 = new
    JButton("9"); JButton b10 = new
    JButton("10");
```

```
frameObj.add(b1); frameObj.add(b2);
frameObj.add(b3); frameObj.add(b4);

frameObj.add(b5); frameObj.add(b6);
frameObj.add(b7); frameObj.add(b8);

frameObj.add(b9);
frameObj.add(b10);
frameObj.setLayout(new
FlowLayout());
frameObj.setSize(300, 300);
frameObj.setVisible(true);
}
public static void main(String argsv[])
{    new FlowLayoutExample();    }
}
```


Individual components: Label

- The object of the Label class is a component for placing text in a container. It is used to display a single line of read only text. The text can be changed by a programmer but a user cannot edit it directly.
- It is called a passive control as it does not create any event when it is accessed. To create a label, we need to create the object of Label class.

AWT Label Class Declaration

public class Label extends Component implements Accessible

AWT Label Fields

1. static int LEFT: It specifies that the label should be left justified.
2. static int RIGHT: It specifies that the label should be right justified.
3. static int CENTER: It specifies that the label should be placed in center.

- Label class Constructors

- Label Class Methods

Button

- A button is basically a control component with a label that generates an event when pushed. The Button class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed.
- When we press a button and release it, AWT sends an instance of ActionEvent to that button by calling processEvent on the button.
- The processEvent method of the button receives the all the events, then it passes an action event by calling its own method processActionEvent. This method passes the action event on to action listeners that are interested in the action

events generated by the button.

- To perform an action on a button being pressed and released, the ActionListener interface needs to be implemented. The registered new listener can receive events from the button by calling addActionListener method of the button.
- The Java application can use the button's action command as a messaging protocol.

AWT Button Class Declaration

public class Button extends Component implements Accessible

Button Class Constructors

- Following table shows the types of Button class constructors

Constructor	Description
Button()	It constructs a new button with an empty string i.e. it has no label.
Button (String text)	It constructs a new button with given string as its label.

• Button Class Methods

Method	Description
void setText (String text)	It sets the string message on the button
String getText()	It fetches the String message on the button.
void setLabel (String label)	It sets the label of button with the specified string.
String getLabel()	It fetches the label of the button.
void addNotify()	It creates the peer of the button.
AccessibleContext getAccessibleContext()	It fetched the accessible context associated with the button.
void addActionListener(ActionListener l)	It adds the specified action listener to get the action events from the button.
String getActionCommand()	It returns the command name of the action event fired by the button.
ActionListener[] getActionListeners()	It returns an array of all the action listeners registered on the button.
T[] getListeners(Class listenerType)	It returns an array of all the objects currently registered as FooListeners upon this Button.
protected String paramString()	It returns the string which represents the state of button.
protected void processActionEvent (ActionEvent e)	It process the action events on the button by dispatching them to a registered ActionListener object.
protected void processEvent (AWTEvent e)	It process the events on the button
void removeActionListener (ActionListener l)	It removes the specified action listener so that it no longer receives action events from the button.
void setActionCommand(String command)	It sets the command name for the action event given by the button.

CheckBox

- The Checkbox class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a Checkbox changes its state from "on" to "off" or from "off" to "on".

AWT Checkbox Class Declaration

public class Checkbox extends Component

implements ItemSelectable, Accessible

- Checkbox Class Constructors

Constructor	Description
Checkbox()	It constructs a checkbox with no string as the label.
Checkbox(String label)	It constructs a checkbox with the given label.
Checkbox(String label, boolean state)	It constructs a checkbox with the given

	label and sets the given state.
Checkbox(String label, boolean state, CheckboxGroup group)	It constructs a checkbox with the given label, set the given state in the specified checkbox group.
Checkbox(String label, CheckboxGroup group, boolean state)	It constructs a checkbox with the given label, in the given checkbox group and set to the specified state.

Method inherited by Checkbox

- The methods of Checkbox class are inherited by following classes:

1. `java.awt.Component`
2. `java.lang.Object`

Method name	Description
<code>void addItemListener(ItemListener IL)</code>	It adds the given item listener to get the item events from the checkbox.
<code>AccessibleContext getAccessibleContext()</code>	It fetches the accessible context of checkbox.
<code>void addNotify()</code>	It creates the peer of checkbox.
<code>CheckboxGroup getCheckboxGroup()</code>	It determines the group of checkbox.

ItemListener[] getItemListeners()	It returns an array of the item listeners registered on checkbox.
String getLabel()	It fetched the label of checkbox.
T[] getListeners(Class listenerType)	It returns an array of all the objects registered as FooListeners.
Object[] getSelectedObjects()	It returns an array (size 1) containing checkbox label and returns null if checkbox is not selected.

Method name	Description
boolean getState()	It returns true if the checkbox is on, else returns off.
protected String paramString()	It returns a string representing the state of checkbox.
protected void processEvent(AWTEvent e)	It processes the event on checkbox.
protected void processItemEvent(ItemEvent e)	It process the item events occurring in the checkbox by dispatching them to registered ItemListener object.
void removeItemListener(ItemListener l)	It removes the specified item listener so that the item listener doesn't receive item events from the checkbox anymore.
void setCheckboxGroup(CheckboxGroup g)	It sets the checkbox's group to the given checkbox.
void setLabel(String label)	It sets the checkbox's label to the string argument.
void setState(boolean state)	It sets the state of checkbox to the specified state.

Choice

- The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits Component class.

AWT Choice Class Declaration

```
public class Choice extends Component  
    implements ItemSelectable, Accessible
```

Choice Class constructor

Constructor	Description
Choice()	It constructs a new choice menu.

Methods inherited by class

- The methods of Choice class are inherited by following classes:

1. `java.awt.Component`
2. `java.lang.Object`

Method name	Description
<code>void add(String item)</code>	It adds an item to the choice menu.
<code>void addItemListener(ItemListener l)</code>	It adds the item listener that receives item events from the choice menu.
<code>void addNotify()</code>	It creates the peer of choice.
<code>AccessibleContext getAccessibleContext()</code>	It gets the accessbile context related to the choice.

String getItem(int index)	It gets the item (string) at the given index position in the choice menu.
int getItemCount()	It returns the number of items of the choice menu.
ItemListener[] getItemListeners()	It returns an array of all item listeners registered on choice.

Method name	Description
T[] getListeners(Class listenerType)	Returns an array of all the objects currently registered as FooListeners upon this Choice.
int getSelectedIndex()	Returns the index of the currently selected item.
String getSelectedItem()	Gets a representation of the current choice as a string.
Object[] getSelectedObjects()	Returns an array (length 1) containing the currently selected item.
void insert(String item, int index)	Inserts the item into this choice at the specified position.
protected String paramString()	Returns a string representing the state of this Choice menu.
protected void processEvent(AWTEvent e)	It processes the event on the choice.
protected void processItemEvent (ItemEvent e)	Processes item events occurring on this Choice menu by dispatching them to any registered ItemListener objects.
void remove(int position)	It removes an item from the choice menu at the given index position.
void remove(String item)	It removes the first occurrence of the item from choice menu.
void removeAll()	It removes all the items from the choice menu.

void removeItemListener (ItemListener l)	It removes the mentioned item listener. Thus it doesn't receive item events from the choice menu anymore.
void select(int pos)	It changes / sets the selected item in the choice menu to the item at given index position.
void select(String str)	It changes / sets the selected item in the choice menu to the item whose string value is equal to string specified in the argument.

List

- The object of List class represents a list of text items. With the help of the List class, user can choose either one item or multiple items. It inherits the Component class.

AWT List class Declaration

```
public class List extends Component  
implements ItemSelectable, Accessible
```

AWT List Class Constructors

Methods Inherited by the List Class

- The List class methods are inherited by following classes:

1. `java.awt.Component`
2. `java.lang.Object`

Method name	Description
<code>void add(String item)</code>	It adds the specified item into the end of scrolling list.
<code>void add(String item, int index)</code>	It adds the specified item into list at the given index position.
<code>void addActionListener(ActionListener l)</code>	It adds the specified action listener to receive action events from list.
<code>void addItemListener(ItemListener l)</code>	It adds specified item listener to receive item events from list.
<code>void addNotify()</code>	It creates peer of list.

void deselect(int index)	It deselects the item at given index position.
AccessibleContext getAccessibleContext()	It fetches the accessible context related to the list.

Method name	Description
ActionListener[] getActionListeners()	It returns an array of action listeners registered on the list.
String getItem(int index)	It fetches the item related to given index position.
int getItemCount()	It gets the count/number of items in the list.
ItemListener[] getItemListeners()	It returns an array of item listeners registered on the list.
String[] getItems()	It fetched the items from the list.
Dimension getMinimumSize()	It gets the minimum size of a scrolling list.
Dimension getMinimumSize(int rows)	It gets the minimum size of a list with given number of rows.
Dimension getPreferredSize()	It gets the preferred size of list.
Dimension getPreferredSize(int rows)	It gets the preferred size of list with given number of rows.
int getRows()	It fetches the count of visible rows in the list.
int getSelectedIndex()	It fetches the index of selected item of list.

<code>int[] getSelectedIndexes()</code>	It gets the selected indices of the list.
<code>String getSelectedItem()</code>	It gets the selected item on the list.
<code>String[] getSelectedItems()</code>	It gets the selected items on the list.
<code>Object[] getSelectedObjects()</code>	It gets the selected items on scrolling list in array of objects.
<code>int getVisibleIndex()</code>	It gets the index of an item which was made visible by method <code>makeVisible()</code>
<code>void makeVisible(int index)</code>	It makes the item at given index visible.
<code>boolean isIndexSelected(int index)</code>	It returns true if given item in the list is selected.
<code>boolean isMultipleMode()</code>	It returns the true if list allows multiple selections.

Method name	Description
protected String paramString()	It returns parameter string representing state of the scrolling list.
protected void processActionEvent(ActionEvent e)	It process the action events occurring on list by dispatching them to a registered ActionListener object.
protected void processEvent(AWTEvent e)	It process the events on scrolling list.
protected void processItemEvent(ItemEvent e)	It process the item events occurring on list by dispatching them to a registered ItemListener object.
void removeActionListener(ActionListener l)	It removes specified action listener. Thus it doesn't receive further action events from the list.
void removeItemListener(ItemListener l)	It removes specified item listener. Thus it doesn't receive further action events from the list.
void remove(int position)	It removes the item at given index position from the list.
void remove(String item)	It removes the first occurrence of an item from list.
void removeAll()	It removes all the items from the list.

<code>void replaceItem(String newVal, int index)</code>	It replaces the item at the given index in list with the new string specified.
<code>void select(int index)</code>	It selects the item at given index in the list.
<code>void setMultipleMode(boolean b)</code>	It sets the flag which determines whether the list will allow multiple selection or not.
<code>void removeNotify()</code>	It removes the peer of list.

Menu

- The object of MenuItem class adds a simple labeled menu item on menu. The items used in a menu must belong to the MenuItem or any of its subclass.
- The object of Menu class is a pull down menu component which is displayed on the menu bar. It inherits the MenuItem class.

AWT MenuItem class declaration

public class MenuItem extends MenuComponent implements Accessible

AWT Menu class declaration

*public class Menu extends MenuItem implements MenuContainer,
Accessible*

Text Field

- The object of a TextField class is a text component that allows a user to enter a single line text and edit it. It inherits TextComponent class, which further inherits Component class.
- When we enter a key in the text field (like key pressed, key released or key typed), the event is sent to TextField. Then the KeyEvent is passed to the registered KeyListener.
- It can also be done using ActionEvent; if the ActionEvent is enabled on

the text field, then the `ActionEvent` may be fired by pressing return key. The event is handled by the `ActionListener` interface.

AWT TextField Class Declaration

```
public class TextField extends TextComponent
```

- **TextField Class constructors**

Constructor	Description
TextField()	It constructs a new text field component.
TextField(String text)	It constructs a new text field initialized with the given string text to be displayed.
TextField(int columns)	It constructs a new textfield (empty) with given number of columns.
TextField(String text, int columns)	It constructs a new text field with the given text and given number of columns (width).

• TextField Class Methods

Method name	Description
<code>void addNotify()</code>	It creates the peer of text field.
<code>boolean echoCharIsSet()</code>	It tells whether text field has character set for echoing or not.
<code>void addActionListener(ActionListener l)</code>	It adds the specified action listener to receive action events from the text field.
<code>ActionListener[] getActionListeners()</code>	It returns array of all action listeners registered on text field.
<code>AccessibleContext getAccessibleContext()</code>	It fetches the accessible context related to the text field.
<code>int getColumns()</code>	It fetches the number of columns in text field.
<code>char getEchoChar()</code>	It fetches the character that is used for echoing.
<code>protected String paramString()</code>	It returns a string representing state of the text field.
<code>protected void processActionEvent(ActionEvent e)</code>	It processes action events occurring in the text field by dispatching them to a registered ActionListener object.
<code>protected void processEvent(AWTEvent e)</code>	It processes the event on text field.
<code>void removeActionListener(ActionListener l)</code>	It removes specified action listener so that it doesn't receive action events anymore.
<code>void setColumns(int columns)</code>	It sets the number of columns in text field.
<code>void setEchoChar(char c)</code>	It sets the echo character for text field.
<code>void setText(String t)</code>	It sets the text presented by this text component to the specified text.

Text Area

- The object of a TextArea class is a multiline region that displays text. It allows the editing of multiple line text. It inherits TextComponent class.
- The text area allows us to type as much text as we want. When the text in the text area becomes larger than the viewable area, the scroll bar appears automatically which helps us to scroll the text up and down, or right and left.

AWT TextArea Class Declaration

public class TextArea extends TextComponent

- **Fields of TextArea Class**

1. `static int SCROLLBARS_BOTH` - It creates and displays both horizontal and vertical scrollbars.
2. `static int SCROLLBARS_HORIZONTAL_ONLY` - It creates and displays only the horizontal scrollbar.
3. `static int SCROLLBARS_VERTICAL_ONLY` - It creates and displays only the vertical scrollbar.
4. `static int SCROLLBARS_NONE` - It doesn't create or display any scrollbar in the text area.

- Class constructors:

Constructor	Description
TextArea()	It constructs a new and empty text area with no text in it.
TextArea (int row, int column)	It constructs a new text area with specified number of rows and columns and empty string as text.
TextArea (String text)	It constructs a new text area and displays the specified text in it.
TextArea (String text, int row, int column)	It constructs a new text area with the specified text in the text area and specified number of rows and columns.
TextArea (String text, int row, int column, int scrollbars)	It constructs a new text area with specified text in text area and specified number of rows and columns and visibility.

- **TextArea Class Methods**

Inner Classes:

Introduction

- In Java, inner class refers to the class that is declared inside class or interface which were mainly introduced, to sum up, same logically relatable classes as Java is purely object-oriented so bringing it closer to the real world.

There are certain advantages associated with inner classes are as follows:

- Making code clean and readable.
- Private methods of the outer class can be accessed, so bringing a new dimension and making it closer to the real world.
- Optimizing the code module.

Syntax of Inner class

```
class Java_Outer_class{  
  //code  
  class Java_Inner_class{  
    //code  
  }  
}
```

Advantage of Java inner classes

1. Nested classes represent a particular type of relationship that is it can access all the members (data members and methods) of the outer class, including private.
2. Nested classes are used to develop more readable and maintainable code because it logically group classes and interfaces in one place only.
3. Code Optimization: It requires less code to write.

Need of Java Inner class

- Sometimes users need to program a class in such a way so that no other class can access it. Therefore, it would be better if you include it within other classes.
- If all the class objects are a part of the outer object then it is easier to nest that class inside the outer class. That way all the outer class can

access all the objects of the inner class.

Types of Nested classes

- There are two types of nested classes non-static and static nested classes. The non-static nested classes are also known as inner classes.

Type	Description
Member Inner Class	A class created within class and outside method.
Anonymous Inner Class	A class created for implementing an interface or extending class. The java compiler decides its name.
Local Inner Class	A class was created within the method.
Static Nested Class	A static class was created within the class.

Nested Interface	An interface created within class or interface.
------------------	---

Member inner class

- A non-static class that is created inside a class but outside a method is

called **member inner class**. It is also known as a **regular inner class**. It can be declared with access modifiers like public, default, private, and protected.

Syntax:


```
class Outer{  
  //code  
  class  
  Inner{  
    //code  
  }  
}
```

Java Member Inner Class Example

- In this example, we are creating a msg() method in the member inner class that is accessing the private data member of the outer class.

```
class TestMemberOuter1{  
    private int data=30; class  
    Inner{  
        void msg(){System.out.println("data is "+data);}  
  
    }  
    public static void main(String args[]){  
        TestMemberOuter1 obj=new TestMemberOuter1();  
        TestMemberOuter1.Inner in=obj.new Inner();  
        in.msg();  
    }  
}
```

}

How to instantiate Member Inner class in Java?

- An object or instance of a member's inner class always exists within an object of its outer class. The new operator is used to create the object of member inner class with slightly different syntax.
- The general form of syntax to create an object of the member inner class is as follows:

Syntax:

*OuterClassReference.**new** MemberInnerClassConstructor();*

Example:

*obj.**new** Inner();*

- Here, OuterClassReference is the reference of the outer class followed by a dot which is followed by the new operator.

Anonymous inner class

- Java anonymous inner class is an inner class without a name and for which only a single object is created. An anonymous inner class can be useful when making an instance of an object with certain "extras" such as overloading methods of a class or interface, without having to actually subclass a class.
- In simple words, a class that has no name is known as an anonymous inner class in Java. It should be used if you have to override a

method of class or interface. Java Anonymous inner class can be created in two ways:

1. Class (may be abstract or concrete).
2. Interface

- **Java anonymous inner class example using class**

```
abstract class Person{  
    abstract void eat();  
}
```

```
class TestAnonymousInner{  
    public static void main(String args[]){  
        Person p=new Person(){  
            void eat(){System.out.println("nice fruits");}  
  
        };  
        p.eat(); }  
}
```

Internal working of given code

```
Person p=new Person(){  
void eat(){System.out.println("nice fruits");}  
};
```

1. A class is created, but its name is decided by the compiler, which extends the Person class and provides the implementation of the eat() method.
2. An object of the Anonymous class is created that is referred to by 'p,' a reference variable of Person type.

Local inner class

- A class i.e., created inside a method, is called local inner class in java. Local Inner Classes are the inner classes that are defined inside a block. Generally, this block is a method body. Sometimes this block can be a for loop, or an if clause.
- Local Inner classes are not a member of any enclosing classes. They belong to the block they are defined within, due to which local inner classes cannot have any access modifiers associated with

them.

- However, they can be marked as final or abstract. These classes have access to the fields of the class enclosing it.
- If you want to invoke the methods of the local inner class, you must instantiate this class inside the method.

Java local inner class example

}

```
public class localInner1{  
    private int data=30;//instance variable void  
    display(){  
        class Local{  
  
            void msg(){System.out.println(data);}  
        }  
        Local l=new Local();  
        l.msg();  
    }  
  
    public static void main(String args[]){  
        localInner1 obj=new localInner1();  
        obj.display();  
    }  
}
```

Rules: Local variables can't be private, public, or protected.

1) Local inner class cannot be invoked from outside the method.

2) Local inner class cannot access non-final local variable till JDK 1.7. Since JDK 1.8, it is possible to access the non-final local variable in the local inner class.

Static inner class

- A static class is a class that is created inside a class, is called a static nested class in Java. It cannot access non-static data members and methods. It can be accessed by outer class name.
 1. It can access static data members of the outer class, including private.
 2. The static nested class cannot access non-static (instance)

data members

Java static nested class example with instance method

```
class TestOuter1{  
    static int data=30;  
    static class Inner{  
        void msg(){System.out.println("data is "+data);}  
  
    }  
    public static void main(String args[]){  
        TestOuter1.Inner obj=new TestOuter1.Inner();  
        obj.msg();  
    }  
  
}
```

- In this example, you need to create the instance of static nested class because it has instance method msg(). But you don't need to create the object of the Outer class because the nested class is static and static properties, methods, or classes can be accessed without

an object.

Collection Framework:

Introduction

- The Collection in Java is a framework that provides an architecture to store and manipulate the group of objects.
- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

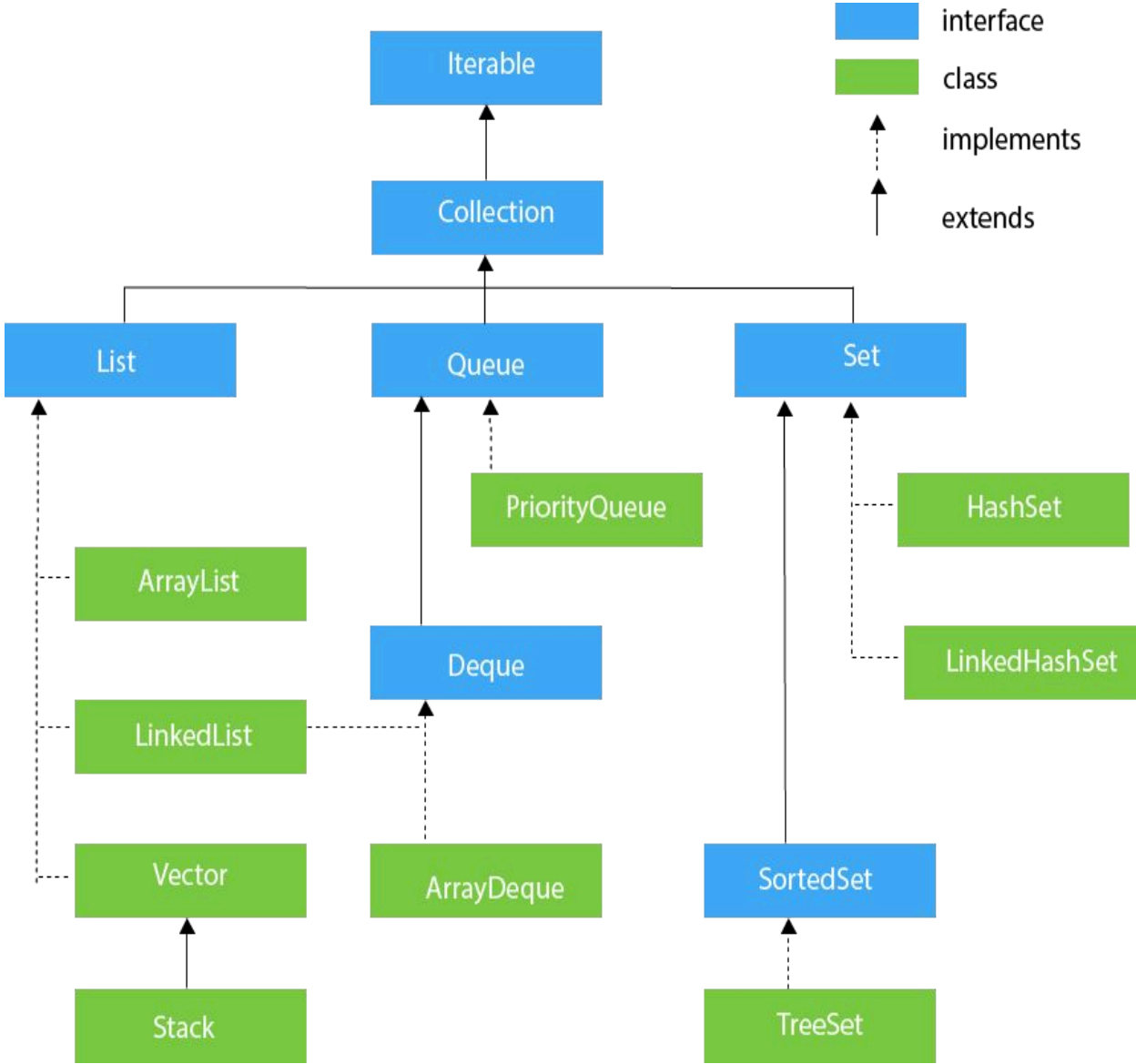
What is Collection in Java

- A Collection represents a single unit of objects, i.e., a group.

What is a framework in Java

- It provides readymade architecture.
- It represents a set of classes and interfaces.
- It is optional.

Constructor	Description
Label()	It constructs an empty label.
Label(String text)	It constructs a label with the given string (left justified by default).
Label(String text, int alignment)	It constructs a label with the specified string and the specified alignment.
Method name	Description
void setText(String text)	It sets the texts for label with the specified text.
void setAlignment(int)	It sets the alignment for label with the specified alignment.



alignment)	
String getText()	It gets the text of the label
int getAlignment()	It gets the current alignment of the label.
void addNotify()	It creates the peer for the label.
AccessibleContext getAccessibleConte xt()	It gets the Accessible Context associated with the label.
protected String paramString()	It returns the string the state of the label.
C o n s t r u	D e s c r i p

c t o r		t i o n	
List()		It constructs a new scrolling list.	
List(int row_num)		It constructs a new scrolling list initialized with the given number of rows visible.	
List(int row_num, Boolean multipleMode)		It constructs a new scrolling list initialized which displays the given number of rows.	
M e t h o d n a m		D e s c r i p t i	

e	o n
void addNotify()	It creates a peer of text area.
void append(String str)	It appends the specified text to the current text of text area.
AccessibleContext getAccessibleContext()	It returns the accessible context related to the text area
int getColumns()	It returns the number of columns of text area.
Dimension getMinimumSize()	It determines the minimum size of a text area.
Dimension getMinimumSize(int rows, int columns)	It determines the minimum size of a text area with the given number of rows and columns.
Dimension getPreferredSize()	It determines the preferred size of a text area.
Dimension preferredSize(int rows, int columns)	It determines the preferred size of a text area with given number of rows and columns.
int getRows()	It returns the number of rows of text

	area.
int getScrollbarVisibility()	It returns an enumerated value that indicates which scroll bars the text area uses.
void insert(String str, int pos)	It inserts the specified text at the specified position in this text area.
protected String paramString()	It returns a string representing the state of this TextArea.
void replaceRange(String str, int start, int end)	It replaces text between the indicated start and end positions with the specified replacement text.
void setColumns(int columns)	It sets the number of columns for this text area.
void setRows(int rows)	It sets the number of rows for this text area.

util Package interfaces

- It contains the collections framework, legacy collection classes, event model, date and time facilities, internationalization, and miscellaneous utility classes (a string tokenizer, a random-number generator, and a bit array).

Hierarchy of Collection Framework

- The **java.util** package contains all the classes and interfaces for the Collection framework.

List

- It found in the java.util package. Let's consider an example for a better understanding where you will see how you can add elements by using a list interface in java.
- List interface in Java is a sub-interface of the Java collections interface. It contains the index-based methods to insert, update, delete, and search the elements.
- It can have duplicate elements also. We can also store the null elements in the list. List preserves the insertion order, it allows positional access and insertion of elements.

}

Example:

```
import
java.util.*;
public class GFG
{
    public static void main(String args[])
    {
        List<String> al = new
        ArrayList<>(); al.add("mango");
        al.add("orange");
        al.add("Grapes");
        for (String fruit :
            al)
            System.out.println(fruit);
    }
}
```

Output :

```
mango
orange
Grapes
```

Set

example for a better

understanding where you will
Output :
ents
ava.

```
[four, one, two, three, five]
```

- The Set follows the unordered way and it found in java.util package and extends the collection interface in java.
- Duplicate item will be ignored in Set and it will not print in the final output. Let's consider an

Example:

```
import java.util.*;
public class SetExample {
    public static void main(String[] args)
    {
        Set<String> Set = new HashSet<String>();
        Set.add("one");
        Set.add("two");

        Set.add("three");
        Set.add("four");
        Set.add("five");
        System.out.println(Set);
    }
}
```

Map

- The Java Map interface, `java.util.Map` represents a mapping between a key and a value.
- More specifically, a Java Map can store pairs of keys and values. Each key is linked to a specific value.
- Once stored in a Map, you can later look

up the value using just the key.

- Let's consider an example for a better understanding where you will see how you can add elements by using the Map interface in java. Let's have a look.

Example:

```
import java.util.*; class MapExample {  
    public static void main(String args[])  
  
    {  
        Map<Integer, String> map =  
new HashMap<Integer,  
String>();  
        map.put(100, "Amit");  
        map.put(101, "Vijay");  
        map.put(102, "Rahul");  
        // Elements can traverse in any  
order for (Map.Entry m :  
map.entrySet()) {  
            System.out.println(m.getKey() + " "  
  
                + m.getValue());  
        }  
    }  
}
```

Output :

```
100 Amit  
101 Vijay  
102 Rahul
```

List

The list interface allows duplicate elements

The list maintains insertion order.

maintain

We can add any number of null values.

Set

Set does not allow duplicate elements.

Set do not

any insertion

o
r
d
e
r
.

But in set almost only one null value.

Map

The map does not allow duplicate elements

The map also does not maintain any insertion order.

The map allows a

single null key at

most and any number
of null values.

List implementation classes are ArrayList, LinkedList.

Set implementation classes are HashSet, LinkedHashSet, and TreeSet.

Map implementation classes are HashMap, Hashtable, TreeMap, ConcurrentHashMap, and LinkedHashMap.

The list provides get() method to get the element at a specified index.

Set does not provide get method to get the elements at a specified index

The map does not provide get method to get the elements at a specified index

If you need to access the elements frequently by using the index then we can use the list

If you want to create a collection of unique elements then we can use set

If you want to store the data in the form of key/value pair then we can use the map.

To traverse the list elements by using ListIterator.

Iterator can be used to traverse the set elements

Through keySet, values, and entrySet.

List interface & its classes

- The List interface in Java provides a way to store the ordered collection. It is a child interface of Collection. It is an ordered collection of objects in which duplicate values can be stored.

Since List preserves the insertion order, it allows positional access and insertion of elements.

- The List interface is found in java.util package and inherits the Collection interface. It is a factory of the ListIterator interface.
- Through the ListIterator, we can iterate the list in forward and backward directions. The

implementation classes of the List interface are ArrayList, LinkedList, Stack, and Vector.

- ArrayList and LinkedList are widely used in Java programming. The Vector class is deprecated since Java 5.

Declaration of Java List Interface

public interface List<E> extends Collection<E> ;

Syntax of Java List

- This type of safelist can be defined as:
List<Obj> list = new ArrayList<Obj> ();



Example of Java List

```
import java.util.*; class
GFG {
public static void main(String[] args)
{
    List<Integer> l1 = new ArrayList<Integer>();
    l1.add(0, 1);
    l1.add(1, 2);
    System.out.println(l1);
    List<Integer> l2 = new ArrayList<Integer>();
    l2.add(1);
    l2.add(2);
    l2.add(3);
    l1.addAll(1, l2);
    System.out.println(l1);
```

```
l1.remove(1);
    System.out.println(l1);
    System.out.println(l1.get(3)
); l1.set(0, 5);
    System.out.println(l1);
}
}
```

Operations in a Java List Interface

- Since List is an interface, it can be used only with a class that implements this interface. Now, let's see how to perform a few frequently used operations on the List.

Operation 1: Adding elements to List class using add() method

Operation 2: Updating elements in List class using set() method

Operation 3: Searching for elements using indexOf(), lastIndexOf methods

Operation 4: Removing elements using remove() method

Operation 5: Accessing Elements in List class using get() method

Operation 6: Checking if an element is present in the List class using
contains() method

Methods of the List Interface

Method	Description
add(int index, element)	This method is used with Java List Interface to add an element at a particular index in the list. When a single parameter is passed, it simply adds the element at the end of the list.
addAll(int index, Collection collection)	This method is used with List Interface in Java to add all the elements in the given collection to the list. When a single parameter is passed, it adds all the elements of the given collection at the end of the list.
size()	This method is used with Java List Interface to return the size of the list.
clear()	This method is used to remove all the elements in the list. However, the reference of the list created is still stored.
remove(int index)	This method removes an element from the specified index. It shifts subsequent elements(if any) to left and decreases their indexes by 1.
remove(element)	This method is used with Java List Interface to remove the first occurrence of the given element in the list.
get(int index)	This method returns elements at the specified index.

set(int index, element)	This method replaces elements at a given index with the new element. This function returns the element which was just replaced by a new element.
-------------------------	--

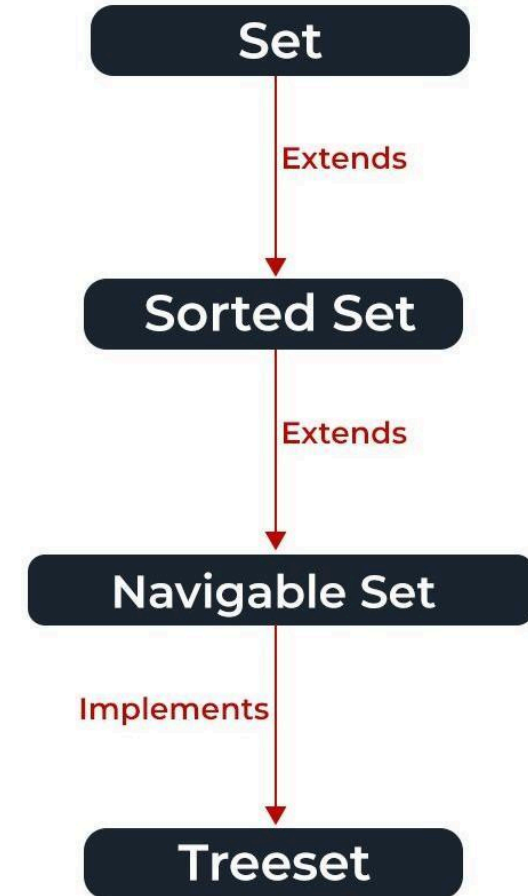
Method	Description
indexOf(element)	This method returns the first occurrence of the given element or -1 if the element is not present in the list.
lastIndexOf(element)	This method returns the last occurrence of the given element or -1 if the element is not present in the list.
equals(element)	This method is used with Java List Interface to compare the equality of the given element with the elements of the list.
hashCode()	This method is used with List Interface in Java to return the hashcode value of the given list.
isEmpty()	This method is used with Java List Interface to check if the list is empty or not. It returns true if the list is empty, else false.
contains(element)	This method is used with List Interface in Java to check if the list contains the given element or not. It returns true if the list contains the element.
containsAll(Collection collection)	This method is used with Java List Interface to check if the list contains all the collection of elements.
sort(Comparator comp)	This method is used with List Interface in Java to sort the elements of the list on the basis of the given comparator .

Set interface & its classes

- The set interface is present in java.util package and extends the Collection interface.
- It is an unordered collection of objects in which duplicate values cannot be stored.
- It is an interface that implements the mathematical set.

- This interface contains the methods inherited from the Collection interface and adds a feature that restricts the insertion of the duplicate elements.
- There are two interfaces that extend the set implementation namely SortedSet and NavigableSet.

- In the image, the navigable set extends the sorted set interface.
- Since a set doesn't retain the insertion order, the navigable set interface provides the implementation to navigate through the Set.
- The class which implements the navigable set is a TreeSet which is an implementation of a self-balancing tree.
- Therefore, this interface provides us with a way to navigate through this tree.



The Set interface is declared as:

public interface Set extends Collection

Creating Set Objects

- Since Set is an interface, objects cannot be created of the type set. We always need a class that extends this list in order to create an object.
- And also, after the introduction of Generics in Java 1.5, it is possible to restrict the type of object that can be stored in the Set. This type-safe set can be defined as:

// Obj is the type of the object to be stored in Set
Set<Obj> set = new HashSet<Obj> ();

- Let us discuss methods present in the Set interface provided below in a tabular format below as follows:

Method	Description
add(element)	This method is used to add a specific element to the set. The function adds the element only if the specified element is not already present in the set else the function returns False if the element is already present in the Set.
addAll(collection)	This method is used to append all of the elements from the mentioned collection to the existing set. The elements are added randomly without following any specific order.
clear()	This method is used to remove all the elements from the set but not delete the set. The reference for the set still exists.
contains(element)	This method is used to check whether a specific element is present in the Set or not.
containsAll(collection)	This method is used to check whether the set contains all the elements present in the given collection or not. This method returns true if the set contains all the elements and returns false if any of the elements are missing.
hashCode()	This method is used to get the hashCode value for this instance of the Set. It returns an integer value which is the hashCode value for this instance of the Set.

Method	Description
isEmpty()	This method is used to check whether the set is empty or not.
iterator()	This method is used to return the <u>iterator</u> of the set. The elements from the set are returned in a random order.
remove(element)	This method is used to remove the given element from the set. This method returns True if the specified element is present in the Set otherwise it returns False.
removeAll(collection)	This method is used to remove all the elements from the collection which are present in the set. This method returns true if this set changed as a result of the call.
retainAll(collection)	This method is used to retain all the elements from the set which are mentioned in the given collection. This method returns true if this set changed as a result of the call.
size()	This method is used to get the size of the set. This returns an integer value which signifies the number of elements.
toArray()	This method is used to form an array of the same elements as that of the Set.

Example:

```
import java.util.*;
public class GFG {
public static void main(String[] args)

{
    Set<String> hash_Set = new HashSet<String>();
    hash_Set.add("Geeks");
    hash_Set.add("For");
    hash_Set.add("Geeks");
    hash_Set.add("Example");
    hash_Set.add("Set");
    // Printing elements of HashSet object

    System.out.println(hash_Set);
}
```

Output

```
[Set, Example, Geeks, For]
```

}

Operations on the Set Interface

- The set interface allows the users to perform the basic mathematical operation on the set. Let's take two arrays to understand these basic operations. Let set1 = [1, 3, 2, 4, 8, 9, 0] and set2 = [1, 3, 7, 5, 4, 0, 7, 5]. Then the possible operations on the sets are:

1. Intersection: This operation returns all the common elements from the given two sets. For the above two sets, the intersection would be:

$$\text{Intersection} = [0, 1, 3, 4]$$

2. Union: This operation adds all the elements in one set with the other. For the above two sets, the union would be:

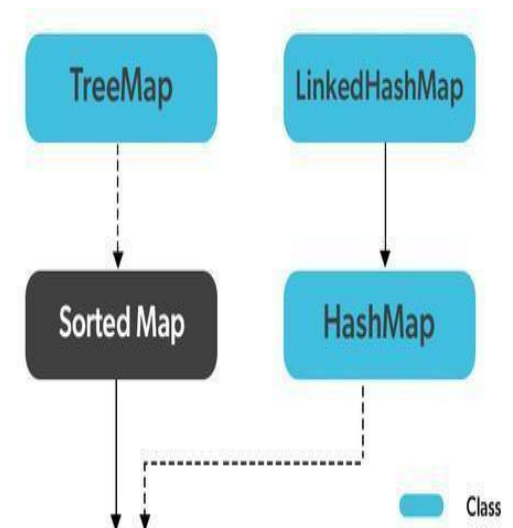
$$\text{Union} = [0, 1, 2, 3, 4, 5, 7, 8, 9]$$

3. Difference: This operation removes all the values present in one set from the other set. For the above two sets, the difference would be:

$$\text{Difference} = [2, 8, 9]$$

Map interface & its classes

- In Java, Map Interface is present in java.util package represents a mapping between a key and a value. Java Map interface is not a subtype of the Collection interface. Therefore it behaves a bit differently from the rest of the collection types. A map contains unique keys.
- Maps are perfect to use for key-value association mapping such as dictionaries. The maps are used to perform lookups by keys or when someone wants to retrieve and update elements by keys. Some common scenarios are as follows:
 1. A map of error codes and their descriptions.



2. A map of zip codes and cities.
3. A map of managers and employees. Each manager (key) is associated with a list of employees (value) he manages.
4. A map of classes and students. Each class (key) is associated with a list of students (value).

Creating Map Objects

- Since Map is an interface, objects cannot be created of the type map. We always need a class that extends this map in order to create an object. And also, after the introduction of Generics in Java 1.5, it is possible to restrict the type of object that can be stored in the Map.

Syntax: Defining Type-safe Map

```
Map hm = new HashMap();
```

```
// Obj is the type of the object to be stored in Map
```

Characteristics of a Map Interface

- A Map cannot contain duplicate keys and each key can map to at most one value. Some implementations allow null key and null values like the HashMap and LinkedHashMap, but some do not like the TreeMap.
- The order of a map depends on the specific implementations. For

example, TreeMap and LinkedHashMap have predictable orders, while HashMap does not.

- There are two interfaces for implementing Map in Java. They are Map and SortedMap, and three classes: HashMap, TreeMap, and LinkedHashMap.

Methods in Java Map Interface

Method	Action Performed
clear()	This method is used in Java Map Interface to clear and remove all of the elements or mappings from a specified Map collection.
containsKey(Object)	This method is used in Map Interface in Java to check whether a particular key is being mapped into the Map or not. It takes the key element as a parameter and returns True if that element is mapped in the map.
containsValue(Object)	This method is used in Map Interface to check whether a particular value is being mapped by a single or more than one key in the Map. It takes the value as a parameter and returns True if that value is mapped by any of the keys in the map.
isEmpty()	This method is used to check if a map is having any entry for key and value pairs. If no mapping exists, then this returns true.
equals(Object)	This method is used in Java Map Interface to check for equality between two maps. It verifies whether the elements of one map passed as a parameter is equal to the elements of this map or not.
get(Object)	This method is used to retrieve or fetch the value mapped by a particular key mentioned in the parameter. It returns NULL when the map contains no such mapping for the key.

put(Object, Object)

This method is used in Java Map Interface to associate the specified value with the specified key in this map.

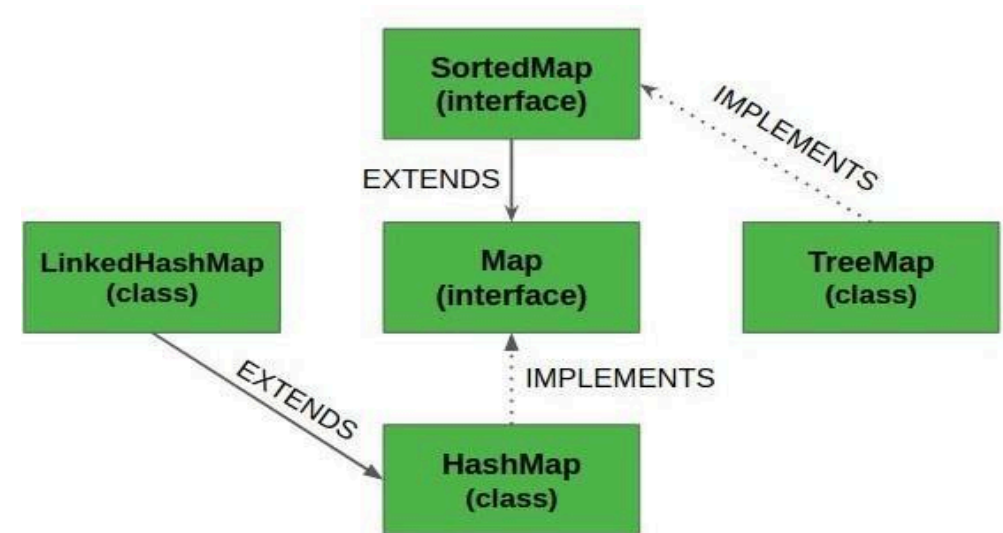
putAll(Map)

This method is used in Map Interface in Java to copy all of the mappings from the specified map to this map.

This method is used in Map Interface to generate a hashCode for the given map containing keys and

keySet()	This method is used in Map Interface to return a Set view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa.
remove(Object)	This method is used in Map Interface to remove the mapping for a key from this map if it is present in the map.
size()	This method is used to return the number of key/value pairs available in the map.
values()	This method is used in Java Map Interface to create a collection out of the values of the map. It basically returns a Collection view of the values in the HashMap.

- Classes that implement the Map interface are depicted in the below media and described later as follows:



MAP Hierarchy in Java

Example:

```
import java.util.*;
public class GFG {
    public static void main(String[] args)
    {
        Map<String, Integer> map = new HashMap<>();
        map.put("vishal", 10);
        map.put("sachin", 30);
        map.put("vaibhav", 20);
        // Iterating over Map
        for (Map.Entry<String, Integer> e : map.entrySet())
            // Printing key-value pairs
            System.out.println(e.getKey() + " "
                               + e.getValue());
    }
}
```

Output

```
vaibhav 20
vishal 10
sachin 30
```

JAVA PRACTICALS

Date:- 04-07-24

Practical no1

Aim:- Write a program to create a class and object in java. Description:-

Class :- A class in Java is a set of objects which shares common characteristics/behaviour and common properties/ attributes. It is a user-defined blueprint or prototype from which objects are created. For example, Student is a class while a particular student named Ravi is an object.

Object:- An object in Java is a basic unit of Object-Oriented Programming and represents real-life entities. Objects are the instances of a class that are created to use the attributes and methods of a class. A typical Java program creates many objects, which as you know, interact by invoking methods.

Source code:-

```
import java.util.Scanner;
```

```
class SimpleInt{
```

```
    S
```

```
    c
```

```
    a
```

```
    n
```

```
    n
```

```
    e
```

```
    r
```

```
    s
```

```
    c
```

```
    ;
```

```
    i
```

```
    n
```

```
    t
```

```
    p
```

```
    ,
```

```
    n
```

```
    ;
```

```
float r,si;
```

```
void getData()  
{  
    sc=new Scanner(System.in);  
    System.out.println("enter your principal  
amount="); p=sc.nextInt();  
    System.out.println("enter number  
of years="); n=sc.nextInt();  
    System.out.println("enter rate if  
interest="); r=sc.nextFloat();
```

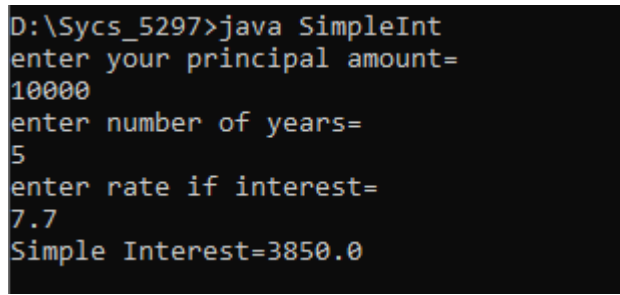
```

}
void display()
{
    si=(p*n*r)/100;
    System.out.println("Simple
    Interest="+si);
}

public static void
main(String args[]){
    SimpleInt s1=new
    SimpleInt(); s1.getData();
    s1.display();}
}

```

Output:-



```

D:\Syics_5297>java SimpleInt
enter your principal amount=
10000
enter number of years=
5
enter rate if interest=
7.7
Simple Interest=3850.0

```

Practice of

addition

import

java.util.Scan

ner; class

abc{

public void

add(int a, int b){

int c;

c=a+b;

System.out.print("addition of two number is =" +c);

}}

public class demo{

public static void main(String args[]){

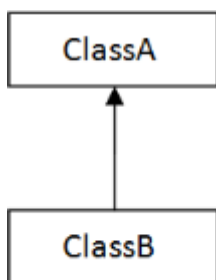
```
abc obj=new abc();  
Scanner sc= new  
Scanner(System.in);  
System.out.println("enter the  
number a="); int a=sc.nextInt();  
System.out.println("enter the  
number b="); int b=sc.nextInt();  
obj.add(a,b);  
}}
```

```
D:\Syics_5297>javac demo.java  
  
D:\Syics_5297>java demo  
enter the number a=  
4  
enter the number b=  
6  
addition of two number is =10
```

Practical no 2

Date :- 11-07-24

Aim:-



1) Single

Single

inheritance

code

Source

code:-

import

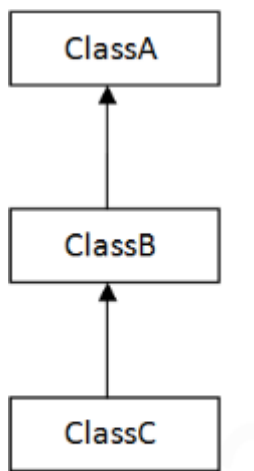
java.io.

*.
;

```
class fruit {  
    public void seed(){  
        System.out.println("I may be with or without seeds");  
    }  
}  
  
class apple  
    extends fruit{  
    public void  
        color(){  
        System.out.println("I am red in colour");  
    }  
}  
  
class market{  
    public static void  
        main(String[]args){ apple  
        a=new apple();  
        a.seed();  
        a.color();  
    }  
}
```

Output:-

```
D:\Syys_5297>java market  
I may be with or without seeds  
I am red in colour
```



2) Multilevel

Multilevel

inheritanc

e Source

code:-

impo

rt

java.

io.*;

class

shap

es{

void

s1(){

System.out.println("welcome to shapes");

}

void s2(){

System.out.println("I am of various type");

}}

class rectangle

extends shapes{ void

r1(){

System.out.println("welcome to rectangle");

}

void area(){

System.out.println(" my area is length into breath");

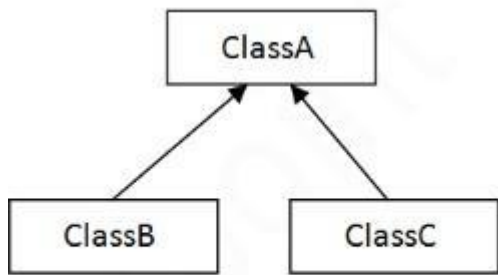
}}


```
class circle extends  
rectangle{ void  
c1(){
```

```
System.out.println("welcome to circle");  
}  
void circumference () {  
System.out.println("others have area and I have circumference");  
}  
}  
class volume {  
public static void  
main(String[] args) { circle  
c=new circle();  
c.s1();  
c.s2();  
c.r1();  
c.area();  
c.c1();  
c.circumference();  
}  
}
```

Output:-

```
D:\Syics_5297>javac multilevel.java  
D:\Syics_5297>java volume  
welcome to shapes  
I am of various type  
welcome to rectangle  
my area is length into breath  
welcome to circle  
others have area and I have circumference
```



3) Hierarchical

Hierarchical

inheritance

Source

code:-

impo

rt

java.

io.*;

class

anim

al{

void

food(

){

System.out.println("We all need food");

}

void shelter(){

System.out.println("We all need shelter");

}

}

class herbivorous

extends animal{ void

h1(){

System.out.println("I am in herbivorous");

}

void grass(){

System.out.println("I

eat grass");

}

}

```
class carnivorous  
extends animal{ void  
c1(){  
System.out.println("I am in class carnivorous");
```

```

}

void f1(){
    System.out.println("I eat herbivorous animals");
}

class omni{
    public static void
    main(String[] args){
        carnivorous c=new
        carnivorous(); c.food();
        c.shelter();
        c.c1();
        c.f1();
        herbivorous h=new
        herbivorous(); h.h1();
        h.grass();
    }
}

```

Output:-

```

D:\Syys_5297>javac hierarchical.java
D:\Syys_5297>java omni
We all need food
We all need shelter
I am in class carnivorous
I eat herbivorous animals
I am in herbivorous
I eat grass

```

Date :- 18-07-24

Practical no 3

Write a java

program bank

Source code:

```

class BankAccount{

```

```

String acc_no;
double d_amt, w_amt , bal;

    BankAccount(String acc_no, double d_amt
,double w_amt){ this.acc_no=acc_no;
this.d_a
mt=d_a
mt;
this.w_a
mt=w_a
mt;
bal=200
0;
}
public void deposit()
{}
public void withdraw()
{}
}
class SavingAccount extends BankAccount{
SavingAccount(String s, double d, double w){
super(s,d,w);
}
public
void
deposit(){
bal+=d_a
mt;
System.out.println("balance"+bal);
}
public void
withdraw(){
if
(bal<=100)
System.out.println("Balance is below 100 , and you cannot
withdraw"); else
bal-=w_amt;

```

```
System.out.println("balance"+bal);}  
}  
public class BankDemo{
```

```

public static void main(String args[]){
    SavingAccount s1=new SavingAccount("a10001356",
    6000, 1000); s1.deposit();
    s1.withdraw();
}
}

```

Output:

```

D:\Syys_5297>javac BankDemo.java
D:\Syys_5297>java BankDemo
balance=8000.0
balance=7000.0

```

```

cl
as
s
Sh
ap
e{
int
l;
int b;
void getArea(int l, int b){}

}
class Rectangle
extends Shape{ void
getArea(int l, int b){
    System.out.println("Area of rectangle is =" +l*b);
}
}
public class Demo1{
    public static void
    main(String args[]){
        Rectangle r= new
        Rectangle();
        r.getArea(10,12);
    }
}

```


}

```
D:\Syys_5297>javac Demo1.java
```

```
D:\Syys_5297>java Demo1  
Area of rectangle is =120
```

Date :- 01-08-24

Exce

ption

Sour

ce

code

:-

```
class InvalidAgeException extends Exception{
```

```
public
```

```
InvalidAgeException(String
```

```
str){ super(str);
```

```
}
```

```
}
```

```
public class
```

```
TestCustomException1{
```

```
static void validate(int
```

```
age) throws
```

```
InvalidAgeException {
```

```
if (age<18){
```

```
throw new InvalidAgeException("age is not
```

```
valid to vote");} else{
```

```
System.out.println("welcome to vote");
```

```
}
```

```
}
```

```
public static void
```

```
main(String str[]){ try{
```

```
validate(13);
```

```

}
catch(InvalidAgeException ex){
System.out.println("Exception caught");
System.out.println("Exception occurred :"+ex);
}
System.out.println("rest of the code");
}
}

```

Output:-

```

D:\Syys_5297>java TestCustomException1
Exception caught
Exception occurred :InvalidAgeException: age is not valid to vote
rest of the code

```

Source code:

```

class table{
synchronized void printTable(int n)
{
for(int i=1;
i<=5; i++){
System.out.
println(n*i);
try{
Thread.slee
p(400);
}catch
(Exception
e){
System.ou
t.println(e);
}
}
}
}
class Mythread1
extends Thread{ table
t;

```

```

Mythrea
d1(table
t){
this.t=t;
}
public
void
run(){
t.print
Table(
5);}
}

class Mythread2
extends Thread{ table
t;
Mythread
2(table t)
{ this.t=t;
}
public
void
run(){
t.print
Table(
100);
}
}

public class
Testsynchronization2{
public static void
main(String args[]){ table
obj=new table();
Mythread1      t1=new
Mythread1(obj);
Mythread2      t2=new
Mythread2(obj);

```

```
t1.start();  
t2.start();  
}  
}
```

Output:-

```
D:\Syics_5297>javac Testsynchronization2.java

D:\Syics_5297>java Testsynchronization2
5
10
15
20
25
100
200
300
400
500
```

Date:- 21-08-24

Practical

no 8

Creating a

package

Source

code:-

package

livingthing

s;

public interface

animal{ public

void

animal_type();

public void

food_eat();

}

Implementing

package in class

package

livingthings;

public class elephant

implements animal{ public void

animal_type(){

System.out.println("Herbivorou
s");

}

public void

```
food_eat(){  
    System.out.println("Grass");  
}  
  
public static void  
main(String args[]){  
    elephant e=new  
    elephant();  
    e.animal_type();  
}
```

```
e.food_eat();  
}  
}
```

Output:-

```
D:\Syys_5297> javac -d . animal.java  
D:\Syys_5297>javac -d . elephant.java  
D:\Syys_5297>java livingthings.elephant  
Herbivorous  
Grass
```

Importing package

livingthings Source

code:-

```
import livingthings.animal;  
  
public class lion  
implements animal {  
    public void animal_type(){  
        System.out.println("Carniv  
ores");  
    }  
  
    public void  
    food_eat(){  
        System.out.println("  
animals ");  
    }  
  
    public static void  
    main(String args[]){ lion  
    li=new lion();  
    li.ani  
    mal_t  
    ype()  
    ;
```


li.foo

d_ea

t());

}

```
}
```

Output:-

```
D:\Syys_5297>javac lion.java
D:\Syys_5297>java lion
Carnivores
animals
```

Networking

Server

source

code:-

import

java.net.*;

import

java.io.*;

class

MyServer

{

public static void main(String args[])throws Exception

{

ServerSocket ss=new

ServerSocket(3333); Socket

s=ss.accept();

DataInputStream din=new

DataInputStream(s.getInputStream()); DataOutputStream

dout=new DataOutputStream(s.getOutputStream());

BufferedReader br=new BufferedReader(new

InputStreamReader(System.in));

String str="",str2="";

while(!str.equals("stop"))

{

str=din.readUTF();

System.out.println("client

says: "+str);

str2=br.readLine();

dout.writeUTF(str2);

```
dout.flush();
```

```

}
din.close();
s.close();
ss.close();
}

}

Client
source
code:-
import
java.net.*
; import
java.io.*;
class
MyClient
{
public static void main(String args[])throws
Exception{ Socket s=new
Socket("localhost",3333);
DataInputStream din=new
DataInputStream(s.getInputStream()); DataOutputStream
dout=new DataOutputStream(s.getOutputStream());
BufferedReader br=new BufferedReader(new
InputStreamReader(System.in));
String
str="",str2="";
while(!str.equals
("stop")){
str=br.readLine()
;
dout.writeUTF(st
r); dout.flush();
str2=din.readUT
F();
System.out.println("Server says: "+str2);

```

```
}  
dout.close();  
s.close();  
}}
```

Output-

```
D:\Sycs_5297>javac MyClient.java
D:\Sycs_5297>java MyClient
Hello Kashish
```

```
D:\Sycs_5297>javac MyServer.java
D:\Sycs_5297>java MyServer
client says: Hello Kashish
```

Date:- 22-08-24

Source

e

code:-

import

java.a

wt.*;

import java.awt.event.*;

public class Evenawt extends Frame implements

ActionListener{ Label a1 ,a2;

TextFiel

d t1;

Button

b;

Evenaw

t(){

super("

Even

odd");

setSize(

500,500

);

setLayout(new

FlowLayout());

setVisible(true);

a1=new Label("Enter a

number"); t1= new

TextField();

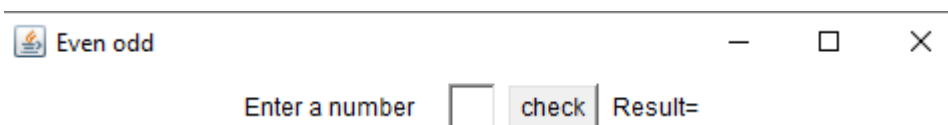
```
b=new  
Button("check"  
); a2=new  
Label("Result=  
"); add(a1);  
add(t1);  
add(b);
```

```

add(a2);
b.addActionLis
tener(this);
}
public void
actionPerformed(ActionEvent a){
int
n=Integer.parseInt(t1.getText());
if(n%2==0)
a2.setText("n
o is even");
else
a2.setText("no is odd");
}
public static void
main(String args[]){
Evenawt ab=new
Evenawt();
}
}

```

Output:-



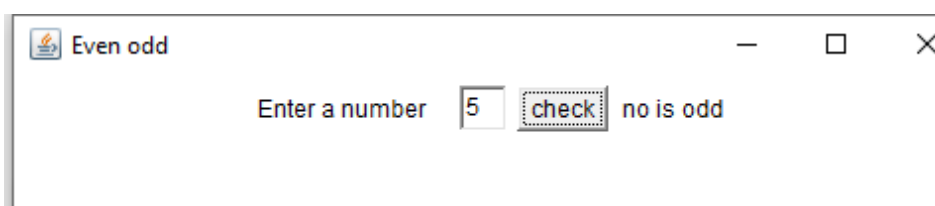
Even odd

Enter a number Result=



Even odd

Enter a number no is even



Even odd

Enter a number no is odd

